100ASK_AM335X

ARM Cortex-A8

Embedded Development Platform

Development Manual

Rev. 1.0

2019/07/01

0755-86200561

support@100ask.net

Shenzhen, Guangdong, China

Shenzhen 100ask Technology Co.

注意事项与售后维修

1. 注意事项

- 使用产品之前，请仔细阅读本手册，并妥善保管，以备将来参考；
- 请注意和遵循标注在产品上的所有警示和指引信息；
- 请使用配套电源适配器，以保证电压、电流的稳定；
- 请在凉爽、干燥、清洁的地方使用本产品；
- 请勿在冷热交替环境中使用本产品，避免结露损坏元器件；
- 请勿在湿气过重、温度过高或过低环境中使用本产品，使用时注意产品的通风；
- 请勿将任何液体泼溅在本产品上，禁止使用有机溶剂或腐蚀性液体清洗本产品；
- 请勿在多尘、脏乱的环境中使用本产品，如果长期不使用，请包装好本产品；
- 请勿在振动过大的环境中使用，任何跌落、敲打或剧烈晃动都可能损坏线路及元器件；
- 请勿在通电情况下，插拔核心板及外围模块(特别是串口模块)；
- 请勿自行维修、拆解本产品，如产品出现故障应及时联系本公司进行维修；
- 请勿自行修改或使用未经授权的配件，由此造成的损坏将不予保修；

2. 售后维修

1) 保修期限

- 底板、核心板：三个月（非人为损坏）
- 显示屏：七天（非人为损坏）

2) 保修说明

- 7 天内：产品（底板、核心板、屏幕）有问题，本公司免费更换/维修，并承担来回运费；
- 7 天至 3 个月内：底板、核心板有问题，本公司免费维修，并承担来回运费（屏幕不提供维修）；
- 3 个月后：底板、核心板有问题，只收元器件价格维修，买家承担来回运费；

3) 联系方式

官方网站：www.100ask.net
淘宝网站：100ask.taobao.com

地　址：广东省深圳市龙岗区布吉南湾街道平吉大道建昇大厦 B1505
联系人：售后维修部
电　话：0755-86200561
邮　编：518114

邮寄须知：保修期限内，寄回本产品请预先垫付邮费，公司不接收任何到付快递。

技术支持与开发定制

1. 技术支持范围

1) 本公司提供的各类开发软件的安装，入门使用，环境搭建；
2) 本公司提供的所有裸机代码的烧写验证；
3) 本公司发布的操作系统的编译、烧写；
4) 本公司发布产品的工控板、模块的硬件原理；
5) 本公司发布的各种外设模块驱动及源码；
6) 本公司发布的配套手册在使用过程中遇到的问题；
7) 本公司产品的故障诊断及售后维修服务；

2. 技术讨论范围

由于嵌入式系统知识范围广泛，涉猎种类繁多，我们无法保证对各种问题都能一一解答，以下内容只提供技术讨论。

1) U-Boot 源码、Linux 内核源码的理解和修改；
2) 非本公司发布的 U-Boot、Linux 内核的编译和移植；
3) 非本公司发布的工控板的各类驱动支持；
4) 非本公司发布的外设模块的硬件原理和驱动设计；

3. 技术支持方式

1) 官方论坛发帖提问(推荐)：bbs.100ask.net
2) 官方淘宝通过阿里旺旺咨询：100ask.taobao.com
3) QQ 群咨询（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）；
4) 技术支持邮箱：support@100ask.net
5) 联系电话：0755-86200561

4. 技术支持时间

星期一到星期五;上午 9:00—12:00;下午 14:00—17:30;

公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至技术支持邮箱或在论坛对应板块发帖，我们将在工作日尽快给您回复。

5. 投诉和建议

如果您对我们有不满意或者建议，可发送邮件到 support@100ask.net 进行反馈，也可拨打 0755-86200561 取得联系，我们将不断改进。

6. 定制开发服务

本公司提供嵌入式操作系统底层驱动、硬件板卡的有偿定制开发服务，以缩短您的产品开发周期。请将需求发送邮件到 support@100ask.net。

资料获取与后续更新

1. 资料的获取

1) 百度网盘下载

百度网盘里面有本产品的所有配套资料，包括原理图、发布的 U-Boot、内核镜像和源码、所需的开发软件、工具等等。

进入 www.100ask.net，在导航栏选择“资料下载”，点击“到百度网盘下载”，跳转到百度网盘后，找到对应的文件夹即可。

2) 视频配套教程

后续会为该工控板录制一套裸机、Linux 驱动、应用的配套付费教学视频，有需要的客户可以进入官方淘宝 100ask.taobao.com 选购。

3) 维基百科教程

维基百科里面会有视频配套的笔记，进入 wiki.100ask.net，选择对应的板块查看。

2. 后续更新

后续文档、视频等资料的更新，为了确保您的资料是最新状态，请密切关注我们的动态，我们将会通过微信公众号和 QQ 群公告推送，购买了本产品的客户请添加 QQ 群（QQ 群号咨询淘宝客服，需提供淘宝购买订单号验证加入）或关注微信公众号。

版权声明

百问科技©2019

深圳百问网科技有限公司版权所有，并保留对本手册及声明的一切权力。

未得到本公司的书面许可，任何单位和个人不得以任何方式或形式对本手册内的任何部分进行复制、摘录、备份、修改、传播、翻译成其他语言、将其全部或部分用于商业用途

更新记录

类别	100ASK_AM335X 系列文档
文档名	100ASK_AM335X Development Manual
当前版本	1.0
适用型号	C3358R512FN512C
编辑	百问科技文档编辑团队
审核	韦东山

修改日志		
版本	修改时间	更改说明
1.0	2019.07.01	初始版本
1.1	2019.09.20	修改虚拟机使用 VmwareWorkstation 15 Player

目录

注意事项与售后维修 I

技术支持与开发定制 II

资料获取与后续更新 III

版权声明 IV

更新记录 V

目录 1

第 1 章 软件资源介绍 3

1.1 bootloader 介绍 3

1.2 linux 内核介绍 3

1.3 根文件系统介绍 4

1.4 开发工具介绍 5

第 2 章 搭建开发环境 6

2.1 使用 vmware 运行虚拟机 6

2.2 安装、使用 MobaXterm 8

2.3 使用 FileZilla 新建 SFTP 连接 14

2.4 安装使用 Source Insight 14

第 3 章 构建系统 21

3.1 获取源码 21

3.2 配置交叉编译工具链 22

3.3 解压编译 bootloader 23

3.4 编译 linux kernel 23

3.5 构建根文件系统 24

第 4 章 烧写系统 26

4.1 Nand Flash 启动/更新系统 26

4.2 SD 卡启动 28

4.3 NFS ROOT 启动 29

第 5 章 平台功能测试 31

5.1 网卡接口测试 31

5.2 USB Host 接口测试.....33
5.3 OTG 接口测试.....34
5.4 耳机接口测试.....37
5.5 LCD 显示测试.....38
5.6 触摸屏校准测试.....39
5.7 屏幕背光调节.....39
5.8 RTC 测试.....40
5.9 按键测试.....41
5.10 LED 灯测试.....42
5.11 串口测试.....42
5.12 RS485 测试.....43
5.13 CAN 功能测试.....44
第 6 章 Qt 应用开发(仅供测试).....47
6.1 安装 Qtcreator.....47
6.2 配置 QtCreator 开发环境.....47
6.3 创建 Helloworld 项目.....50
6.4 编译 Qt 程序.....53
6.5 在开发板运行 Qt 程序.....55
附录 56

第1章软件资源介绍

1.1 bootloader 介绍

Bootloader 是在操作系统运行之前运行的一段代码，用于引导操作系统。通常每个操作系统都有一组专属的引导加载程序。引导加载程序通常可以通过多种方式引导操作系统内核，还有各种命令用于调试或修改内核运行环境。

bootloader有许多版本，这里我们选择使用Das U-Boot（“Universal BootLoader”，通常缩写为U-Boot）是一个开源的主引导加载程序，用于引导设备的操作系统内核，并含有多种命令以便调试系统。它适用于多种计算机体系结构，包括 68k，ARM，Blackfin，MicroBlaze，MIPS，Nios，SuperH，PPC，RISC-V 和 x86。

- U-boot 官网 https://www.denx.de/wiki/U-Boot
- 源码下载页面 http://ftp.denx.de/pub/u-boot/
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask-am335x_uboot.git

百问网提供的 U-Boot 有如下特性：

特性	是否支持	作用	是否开源
mmc 启动	支持	通过 sd 卡启动系统	Yes
Nand 启动	支持	通过板载 nandflash 启动系统	Yes
网卡启动	支持	通过网络挂载方式启动系统	Yes
usb 启动	支持	通过 usb 设备启动	Yes

注意: u-boot 编译方法请参考 configbootloader

1.2 linux 内核介绍

Linux 内核（英语：Linux kernel）是一种开源的类 Unix 操作系统宏内核。整个 Linux 操作系统家族基于该内核部署在传统计算机平台（如个人计算机和服务器，以 Linux 发行版的形式）和各种嵌入式平台，如路由器、无线接入点、专用小交换机、机顶盒、FTA 接收器、智能电视、数字视频录像机、网络附加存储（NAS）等。工作于平板电脑、智能手机及智能手表的 Android 操作系统，它的底层操作系统也是 Linux。尽管在桌面计算机的占用率较低，但基于 Linux 的操作系统统治了几乎从移动设备到主机的其他全部领域。实际 Linux 的发行版 Ubuntu，其易用性也逐渐接近 Windows。

- Linux kernel 官网 https://www.kernel.org/
- linuxKernel 维基百科 https://www.wiki.kernel.org/
- 在线阅读 linuxkernel 源码 https://elixir.bootlin.com/
- TI 官方 linuxkernel 参考文档 http://processors.wiki.ti.com/index.php/Linux_Kernel_Users_Guide
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask_am3358_kernel.git

上述 Git 仓库是专为 100ask 335x 系列开发板制定的 Linux 内核，它有如下特性：

名称	是否支持	作用	是否开源
USB HOST	支持	USB Host 驱动，支持 OHCI 和 EHCI 两种传输模式	Yes
USB OTG	支持	Usb otg 驱动，支持 device 模式和 host 模式	Yes
Network Port	支持	RJ-45 以太网驱动	Yes
MMC/SD	支持	MMC/SD 卡驱动	Yes

EMMC	支持	EMMC 驱动	Yes
I2C	支持	I2C 驱动	Yes
SPI	支持	SPI 驱动	Yes
LCD	支持	LCD 屏驱动，默认 4.3 寸 272*480 液晶屏	Yes
RTC	支持	内置 RTC 时钟驱动	Yes
ADC	支持	ADC 驱动	Yes
TouchScreen	支持	电容触摸	Yes
UART	支持	串口驱动	Yes
PMU	支持	电源管理驱动	Yes
LED	支持	LED 驱动，包括 GPIO LED 和 PWM	Yes
Button	支持	GPIO Button 驱动	Yes
Camera	支持	摄像头驱动	Yes

注意：内核编译方法请参考编译 linux kernel

1.3 根文件系统介绍

Buildroot 是一组 Makefile 和补丁，可简化并自动化地为嵌入式系统构建完整的、可启动的 Linux 环境(包括 bootloader、Linux 内核、包含各种 APP 的文件系统)。Buildroot 运行于 Linux 平台，可以使用交叉编译工具为多个目标板构建嵌入式 Linux 平台。Buildroot 可以自动构建所需的交叉编译工具链，创建根文件系统，编译 Linux 内核映像，并生成引导加载程序用于目标嵌入式系统，或者它可以执行这些步骤的任何独立组合。例如，可以单独使用已安装的交叉编译工具链，而 Buildroot 仅创建根文件系统。

参考网址

- Buildroot 用户手册 https://buildroot.org/downloads/manual/manual.html
- BuildRoot 源码下载位置 https://buildroot.org/downloads/
- 目录结构的位置以及作用请参考网址 http://wiki.100ask.net/Buildroot
- Git 仓库地址 https://git.dev.tencent.com/weidongshan/100ask_am3358_buildroot.git

百问网提供的 Buildroot 有如下特性：

文件系统支持	作用
web 环境	html+php 网络开发环境
Qt 环境	Qt 5.9 开发环境（选用）
Vim 开发环境	vim8.0 环境
串口传输文件	通过 z-mode 协议使用串口传输文件到开发板
ssh 工具	通过 ssh 工具登录开发板系统
nfs 工具	通过 nfs 命令挂载主机文件
电子相册	在 LCD 上浏览图片，支持放大缩小等
Arduino 编程	在 windows 下对开发板进行模块编程

注意：根文件系统源码下载及编译方法请参考构建根文件系统

1.4 开发工具介绍

名称	位置	软件作用
arduino-1.8.9-windows.exe	01_tools\	Windows 下编译开发环境，主要适用模块应用开发
FileZilla_3.22.1_win32-setup.exe	01_tools\	Windows 下 ftp 工具，传输文件到 Ubuntu 系统中
HPUSBDiskStorage_Format_Tool 2.0.6.EXE	01_tools\	Windows 下格式化 sd 卡工具，主要适用于在更新 nandflash 的格式化。
MobaXterm_Portable_v11.0.zip	01_tools\	Windows 下 ssh 工具;ftp 工具;串口工具。
npp.7.6.1.Installer.exe	01_tools\	Window 下常用的文本阅读工具
qt-creator-opensource-linux-x86_64-4.8.0.run	01_tools\	Linux 下 qt 应用程序开发客户端
SD_CardFormatter5.0.1Setup.exe	01_tools\	Windows 下 sd 卡格式化工具，用于 sd 卡启动
sourceinsight4098-setup.exe	01_tools\	Windows 下源码阅读工具
VMware-workstation-full-15.1.0-13591040.exe	01_tools\	Windows 下系统虚拟化工具，用于虚拟化 ubuntu 18.04 系统
win32diskimager-1.0.0-install.exe	01_tools\	Windows 下开发板系统镜像烧写工具，用于 sd 卡启动

第2章 搭建开发环境

本章主要讲解如何使用 Vmware workstation 加载百问网制作的 ubuntu18.04 虚拟机镜像，并使用 MobaXterm 工具与 Ubuntu 建立 ssh 连接、FTP 连接以及串口连接；介绍代码阅读工具 Sourceinsight 如何新建一个内核项目，最后是关于 ubuntu18.04 的一些配置说明

本章所需工具

- VMware-workstation-full-15.1.0-13591040.exe
- sourceinsight4098-setup.exe
- MobaXterm_Portable_v11.0.zip

2.1 使用 vmware 运行虚拟机

2.1.1 安装wmwareWorkstationPlayer

Windows 下有很多虚拟机软件，目前市面上流行的有 VMware 和 VirtualBox。VMware 分为收费专业版 Workstation Pro 和非商用免费版 Workstation Player，推荐使用 Workstation Player。

首先从 VMware 官网（www.vmware.com）下载 Workstation Player 安装包或者使用我们提供的安装包(目录在资料光盘\01_tools 目录下)，这里使用资料光盘安装包：VMware-workstation-full-15.1.0-13591040.exe。

VMWare 安装完成后，有两个软件，它们都可以使用，建议使用第 2 个：

1. Vmware Workstation Pro：这是收费的，可以试用 30 天。
2. Vmware Workstation 15 Player：这是免费的。

百问网团队特意根据本节内容录制了 1 节视频:

腾讯视频在线观看：https://v.qq.com/x/page/w05186fncbn.html

注明：如想自己配置 ubuntu 等开发环境请参考 http://wiki.100ask.net/Configuring_ubuntu

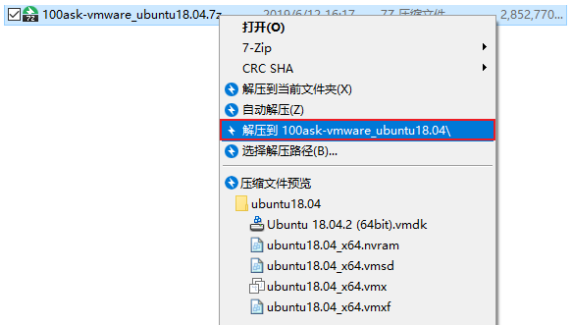
2.1.2 使用虚拟机打开Ubuntu

解压资料光盘提供的虚拟机压缩包(资料光盘\01_tools\100ask-vmware_ubuntu18.04.7z) 如图 2.1.2.1 所示。

注意:

- 100ask-vmware_ubuntu18.04 不支持 **win7 32** 位和 **Windows xp** 电脑系统，有疑问请联系技术支持。
- 100ask-vmware_ubuntu18.04 用到了 **Virtualization Technology(VT)**技术,请提前把电脑的 VT 打开，由于不同的电脑打开方法不一样，所以具体的打开方法请百度，比如在百度搜“联想电脑 开启 Virtualization Technology”。

1) 运行 Vmware Workstation 15 player，在欢迎界面选中”**打开虚拟机**”，如图 2.1.2.2 所示。



图解压虚拟机

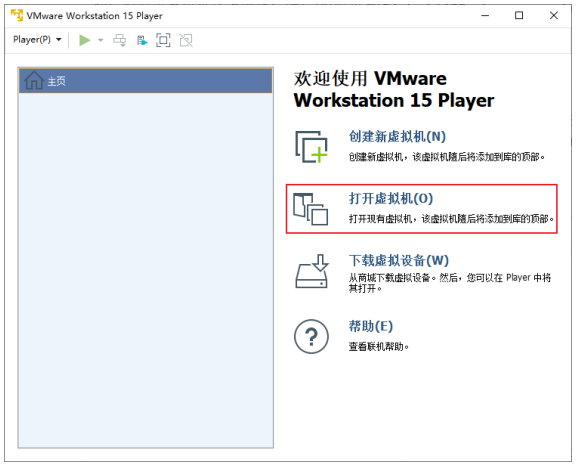


图 2.1.2.2 打开虚拟机

2) 在弹出的窗口中，找到前面解压的 Ubuntu，选中“**ubuntu-18.04_x64.vmx**”,再单击“**打开**”如图 2.1.2.3 所示。再单击“**开启此虚拟机**”，如图 2.1.2.4 所示。

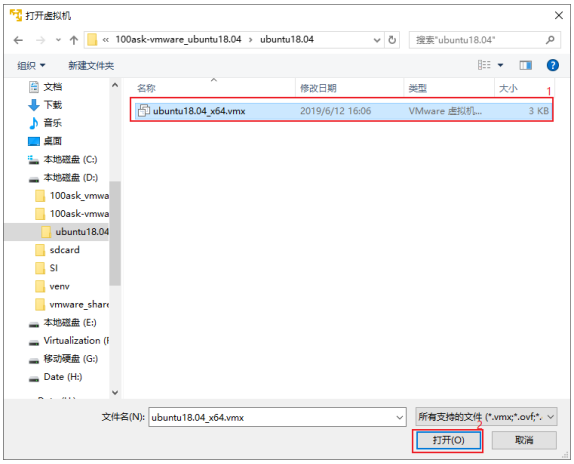


图 2.1.2.3 选中 Ubuntu

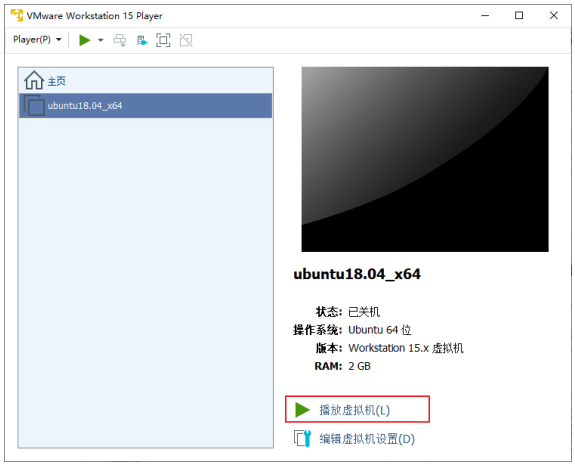


图 2.1.2.4 播放虚拟机

3) 第一次打开会有如 2.1.2.5 所示的提示，选择默认的“**我已复制该虚拟机**”。此时 VMware Workstation 15 player 开始启动虚拟机，等待一会后出现如图 2.1.2.6 所示的登陆界面，在“**Password**”中输入密码“**123456**”回车（注意：虚拟机默认没有开启小键盘，如果使用小键盘输入，请先开启小键盘）。

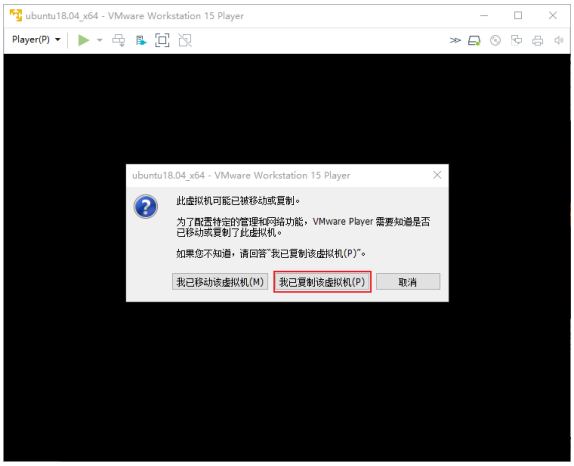


图 2.1.2.5 我已复制该虚拟机

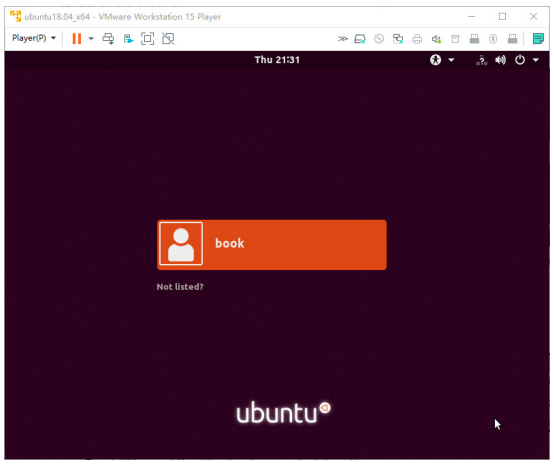


图 2.1.2.6 登陆界面

2.2 安装、使用 MobaXterm

MobaXterm 进行远程操作计算机的终极工具箱。在单个 Windows 应用程序中，它提供了大量功能，这些功能是为程序员、网站管理员、IT 管理员以及需要以更简单的方式处理远程作业的所有用户量身定制的。MobaXterm 向 Windows 桌面提供所有重要的远程网络工具（SSH，X11，RDP，VNC，FTP，MOSH .....）和 Unix 命令（bash，ls，cat，sed，grep，awk，rsync）等，仅仅需要一个 exe 文件即可。

参考网址

- 官网 https://mobaxterm.mobatek.net/
- 使用文档 https://mobaxterm.mobatek.net/documentation.html

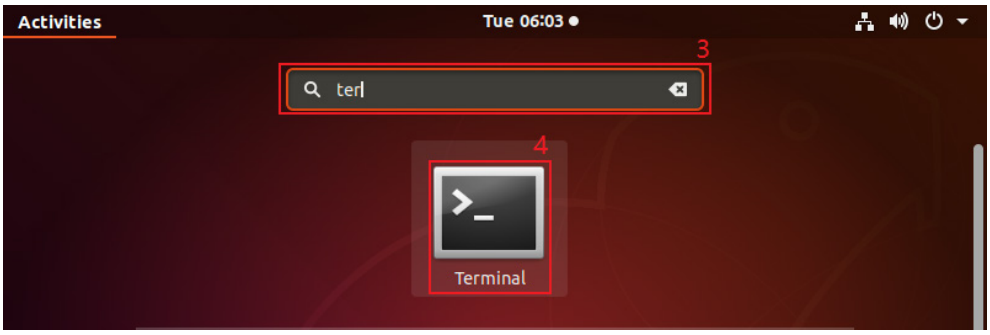
2.2.1 安装及获取ubuntuIP地址

解压文件“MobaXterm_Portable_v11.0.zip”即可。第一次打开会自解压比较慢，后续就正常了。

使用 MobaXterm 是为了远程登录 Vmware 里的 Ubuntu，所以需要先知道 Ubuntu 的 IP。方法如下图 2.2.1.1 所示。点击 Ubuntu 桌面左上角图标，输入“**term**”可以得到图中蓝框中的“**Terminal**”程序，运行它；然后执行“**ifconfig**”命令即可查看 Ubuntu 的 IP，如图 2.2.1.2 所示。



图 2.2.1.1 打开 Terminal



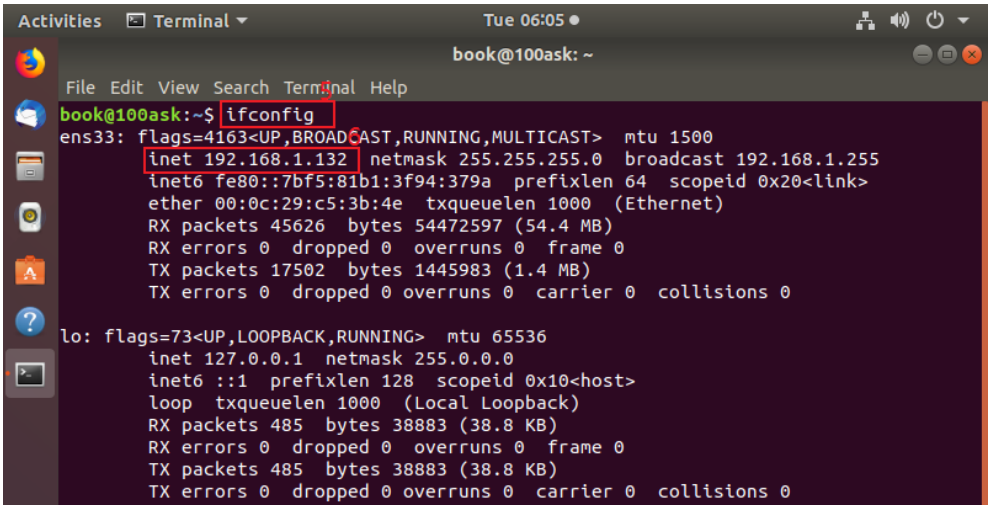


图 2.2.1.2 查看 Ubuntu 的 IP

注意：本文档假设 Ubuntu 的 IP 为 192.168.1.132，在上图中网卡为 ens33(**你的电脑上网卡名也许不一样**)。

2.2.2 新建SSH连接

1) 打开 MobaXterm，点击左上角的 **Session**（会话控制），在弹出的窗口中选择 SSH，如图 2.2.2.1 和 2.2.2.2 所示。

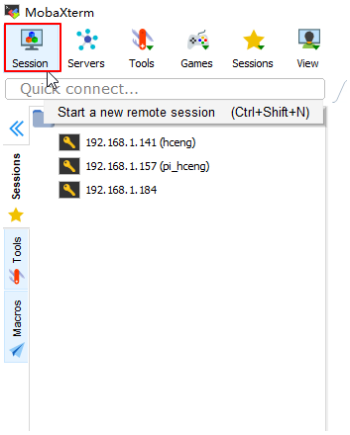


图 2.2.2.1 选择 Session

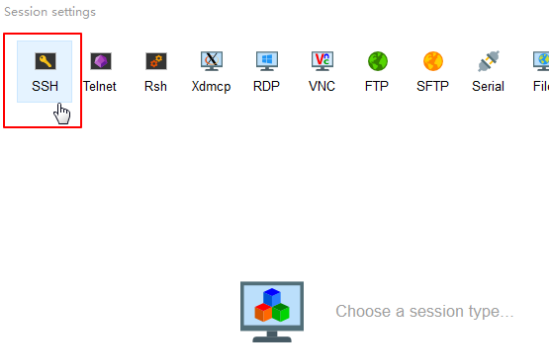


图 2.2.2.2 选择 SSH

2) 在弹出操作框里输入 Ubuntu 的 IP 和端口号（默认是 22），然后点击“**OK**”，如图 2.2.2.3 所示。

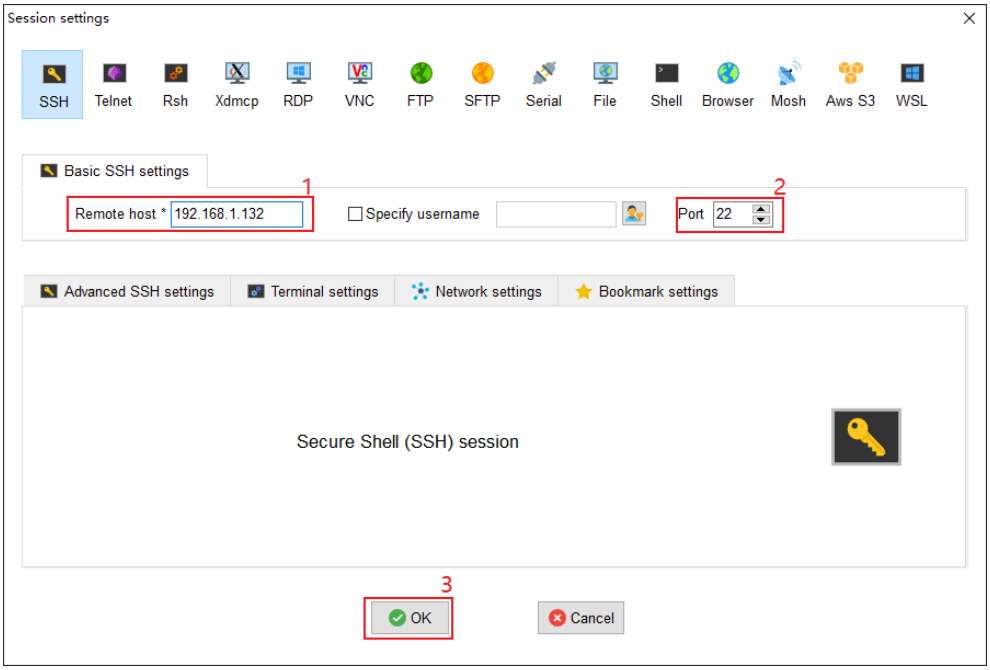


图 2.2.2.3 SSH 选项

3) 在新窗口中输入账号“**book**”和密码“**123456**”，再按下键盘“回车键”登陆 Ubuntu。此时界面分为两块，左边的是主机的文件，右边是终端。勾选左下角的“**Follow terminal folder**”可以让它们的工作路径保持一致，如图 2.2.2.4 所示。

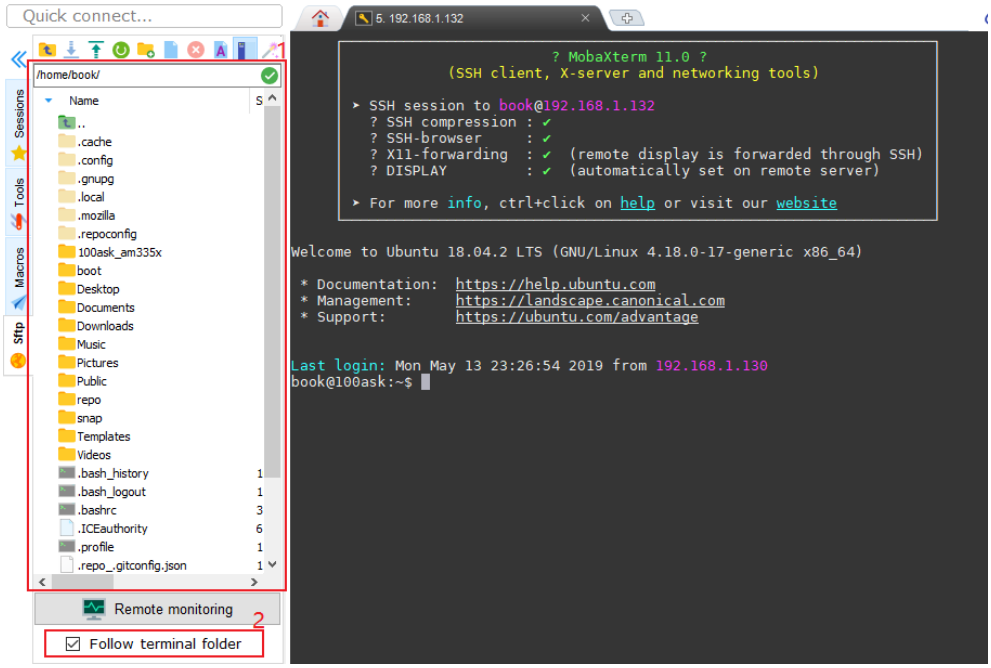


图 2.2.2.4 SSH 工作窗口

2.2.3 新建串口连接

在后面的操作里，都是通过串口与板子进行“交流”。串口是串行接口的简称，是指数据一位一位地顺序传送，其特点是通信线路简单。

在电脑上安装好 MobaXterm 后，接上 USB 串口模块，并跟开发板连好线。

在 MobaXterm 里敲打键盘，就会通过 USB 串口模块，将数据经过 RS232 延长线传给板子，板子就能接收到我们在电脑上发送的数据。

反过来，板子发送的数据首先经过 RS232 延长线到 USB 串口模块，MobaXterm 读取数据后显示出来。

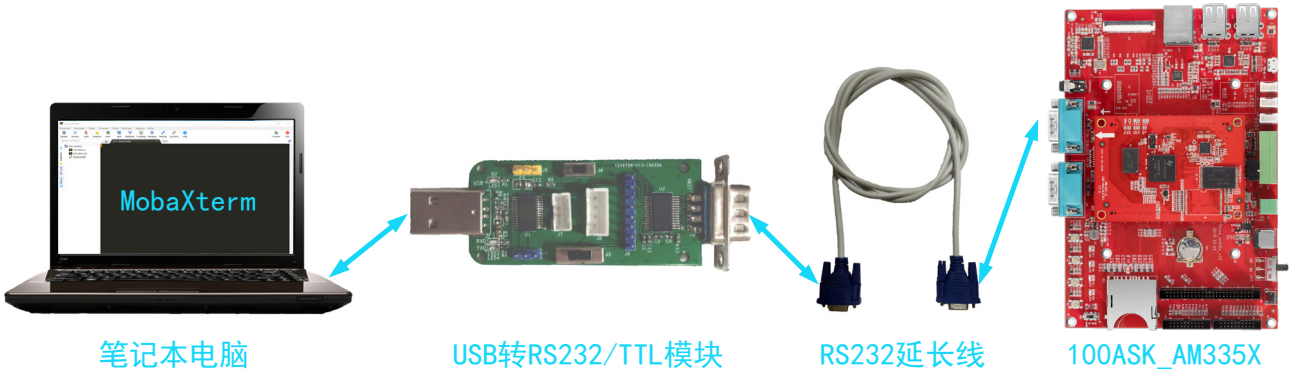


图 2.2.4.1 示意图

1) 安装 USB 串口模块驱动

将 USB 转 RS232/TTL 串口模块插在电脑 USB 上，此时 Windows 会自动安装驱动(安装可能比较慢，等一分钟左右)。打开电脑的“设备管理器”，在“端口 (COM 和 LPT)”项下，可以看到如图 2.2.4.2 中的“USB Serial Port(COM3)”。这里的“**COM3**”可能与你电脑上的不一样，记住你电脑显示的数字。

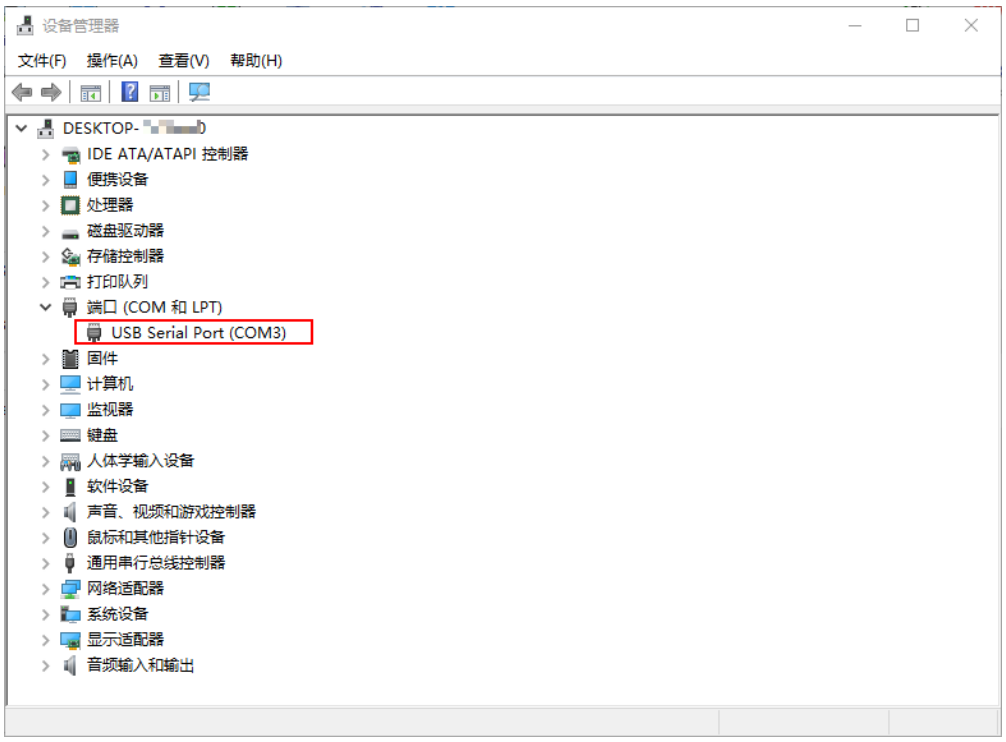


图 2.2.4.2 设备管理器查看串口端口号

如果电脑没有显示出端口号，就需要手动安装驱动，从驱动精灵官网（www.drivergenius.com）下载一个驱动精灵，安装、运行、检测，会自动安装上串口驱动。

2) 连线、配置

首先如图 2.2.4.3 所示将串口模块与电脑、板子连接。

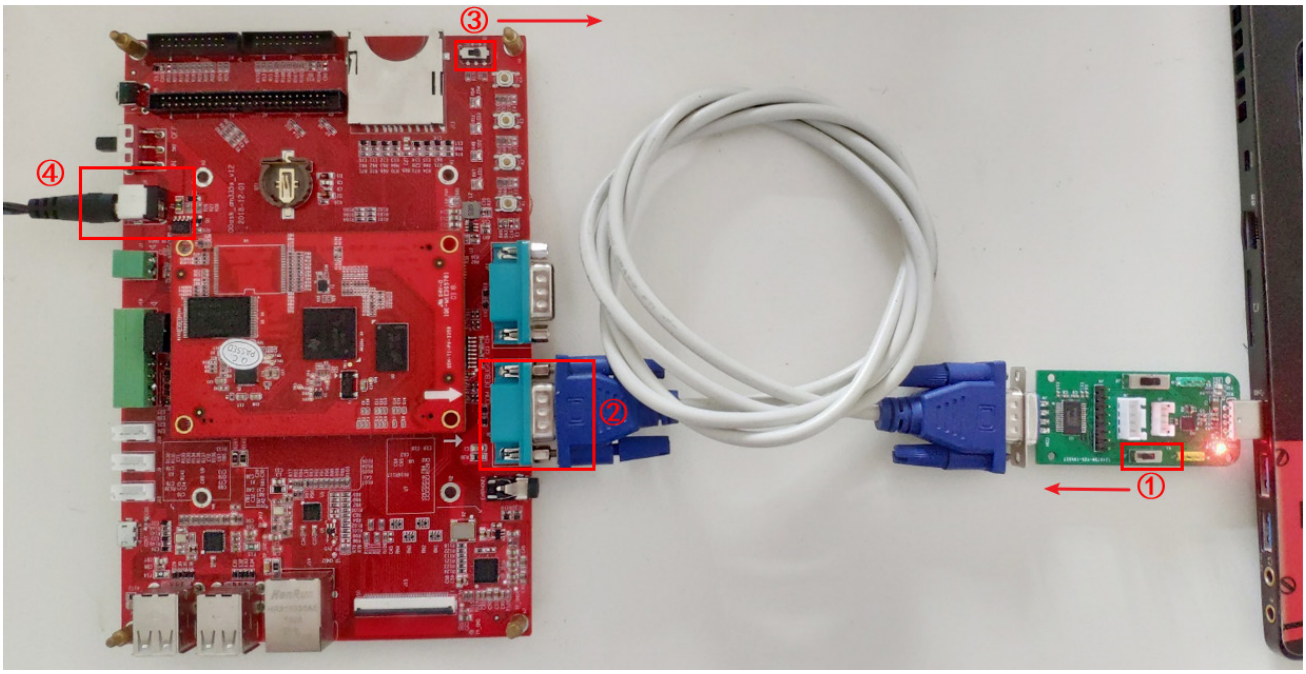


图 2.2.4.3 串口模块连接实物图

其中特别需要注意的几点：

a) 串口模块的电平选择开关拨到如图所示的左边，以选择 RS232 电平；
b) RS232 延长线连接板子如图所示下方的 RS232 接口，下方的才是默认的 debug 接口；
c) 板子的启动选择拨到如图所示的右边，以选择从 Nand Flash 启动；
d) 板子如图所示插上配套的电源，电源开关暂时不用打开；

打开 MobaXterm，点击左上角的“Session”，在弹出的界面选中“Serial”，如图 2.2.4.4 所示选择端口号（前面设备管理器显示的端口号 COM3）、波特率（Speed 115200）、流控（Flow Control: none）,最后点击“OK”即可。

注意：流控（Flow Control）一定要选择 none，否则你将无法在 MobaXterm 中向串口输入数据。

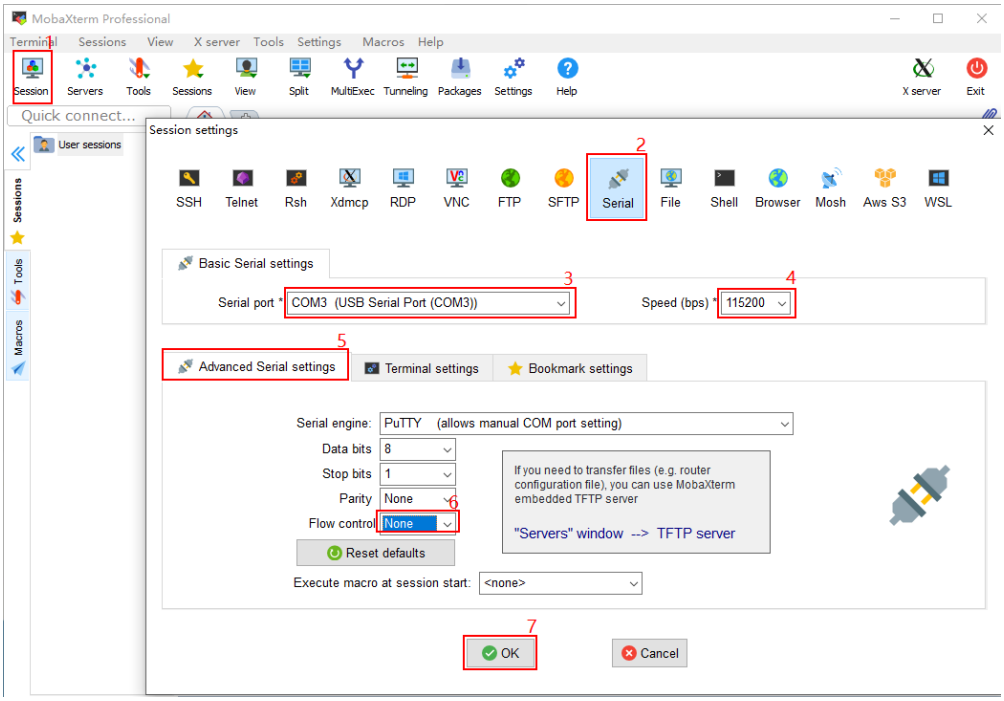


图 2.2.4.4 MobaXterm 串口对话设置

随后显示一个黑色的窗口， 此时打开板子的电源开关，将收到板子串口发过来的数据，如图 2.2.4.5 所示。

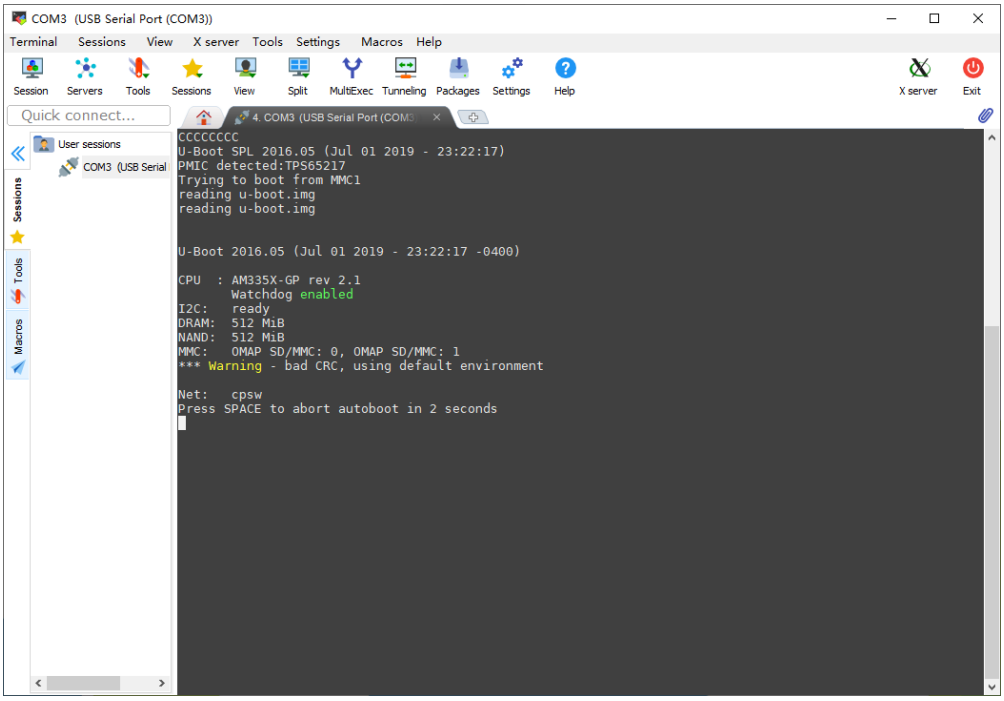


图 2.2.4.5 接收板子串口发送的数据

2.3 使用 FileZilla 新建 SFTP 连接

MobaXterm 支持 FTP 和 SFTP 连接。FTP 安全性没有 SFTP 好，但速度比 SFTP 快，可根据自己需求选择适当的协议。MobaXterm 的 FTP、SFTP 有时不稳定，我们推荐使用另一个软件 FileZilla 来传输文件。

注意：我们提供的 ubuntu 中没有安装 FTP 服务，你只能使用 SFTP 服务；后文提到的 TFTP 服务，跟 FTP 服务是不一样的，U-Boot 可以使用 TFTP 从 Ubuntu 下载文件。

1) 使用 FileZilla 软件来实现 SFTP 文件传输：

安装、运行 FileZilla 后，如图 2.2.3.1tu 的 IP（192.168.1.132、用户名（book）、端口号（22），单击“快速链接”即可。

注意：端口号必须填入 22，它对应 SFTP 服务。

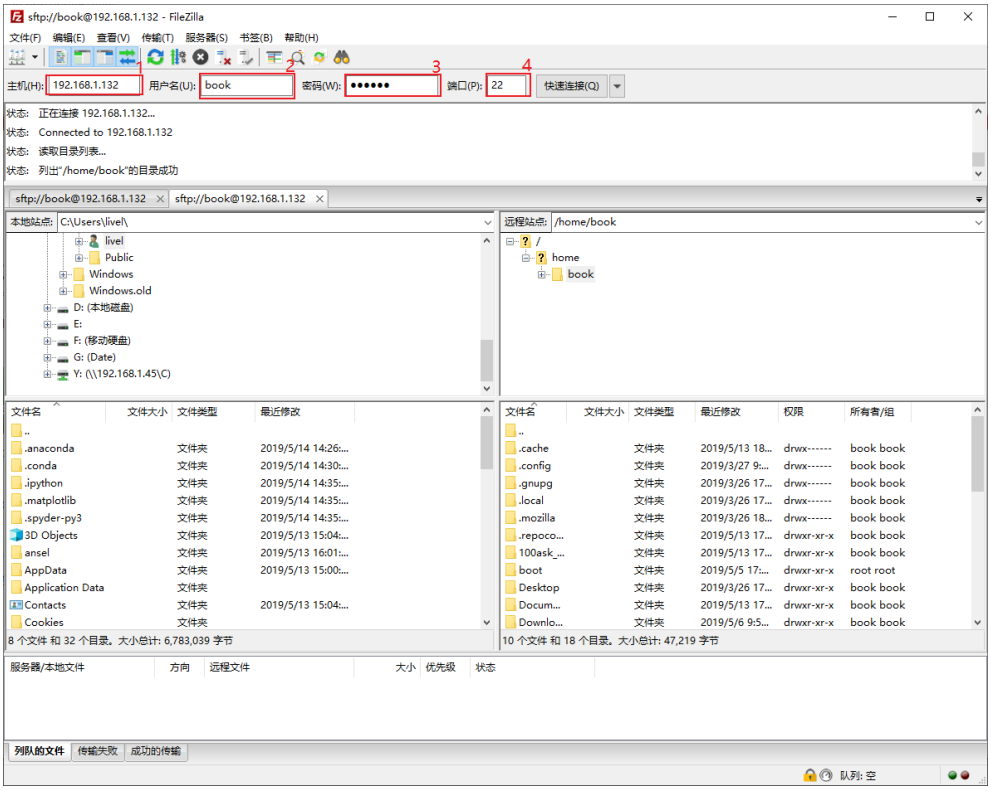


图 2.3.1 FileZilla 连接成功

2.4 安装使用 Source Insight

2.4.1 Source Insight简介

Source Insight 是 Source Dynamics 公司出品的源代码编辑器。Source Insight 提供语法突出显示，代码导航和可自定义的键盘快捷键。它不仅仅是一个编辑器，而是一个理解大型源代码库的工具，因此被称为“程序编辑器和分析器”。它灵活轻便，提供有用的功能，如关系，上下文和符号窗口。它在建源码工程时，构建了符号信息的内部数据库，所以还可以显示引用树，类继承图和调用树。它的最大好处是加快了对不熟悉项目的代码理解。

参考网址

- 官网主页 https://www.sourceinsight.com/

- 软件下载页面 https://www.sourceinsight.com/trial/
- 用户使用教程 https://www.sourceinsight.com/doc/v4/userguide/index.html

2.4.2 安装并新建项目示例

安装并运行(资料光盘\ 01_tools)中的 sourceinsight4098-setup.exe 文件。
这里我们新建一个 linux kernel 的 source Insight 为例对 source Insight 使用进行说明

1) 在桌面上找到 source Insight 双击运行，点击 **Project->New Project**,如图 2.3.2.1 所示

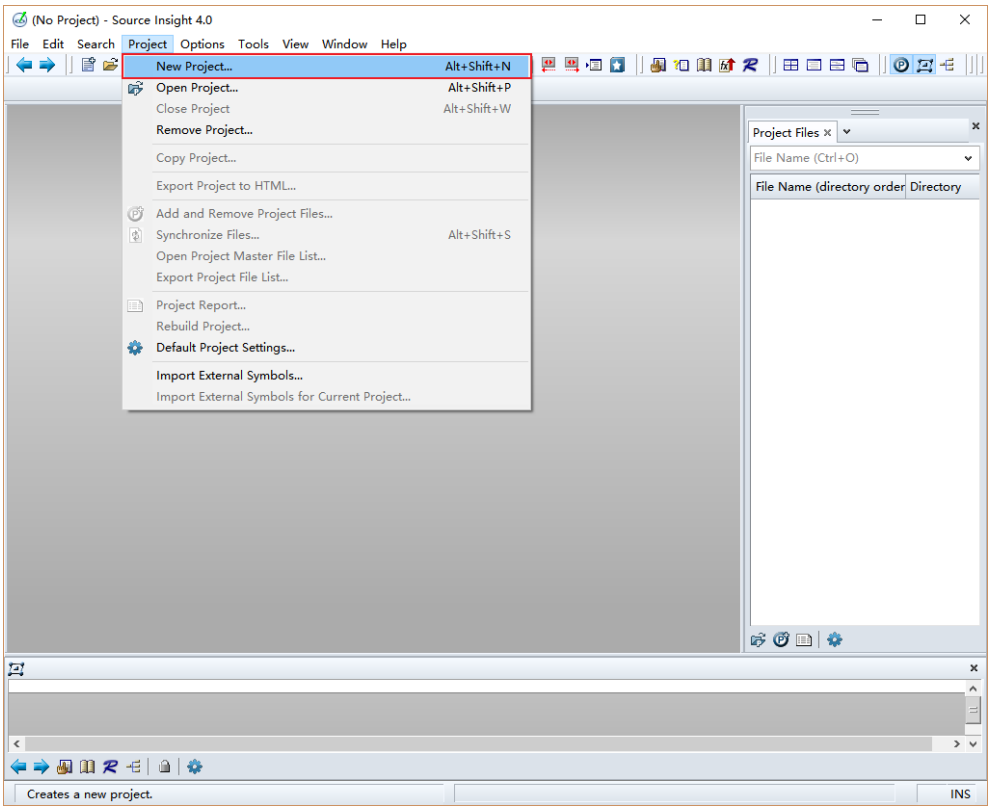


图 2.3.2.1 选择 New Project

2) 在弹出的 New Project 对话框中设置 **New project name**(项目的名称)让后设置 **Where do you want to store the project data file?** (项目文件保存位置)，点击 **Browse…**选择所在保存位置，设置好以后点击 OK 如图 2.3.2.2 所示

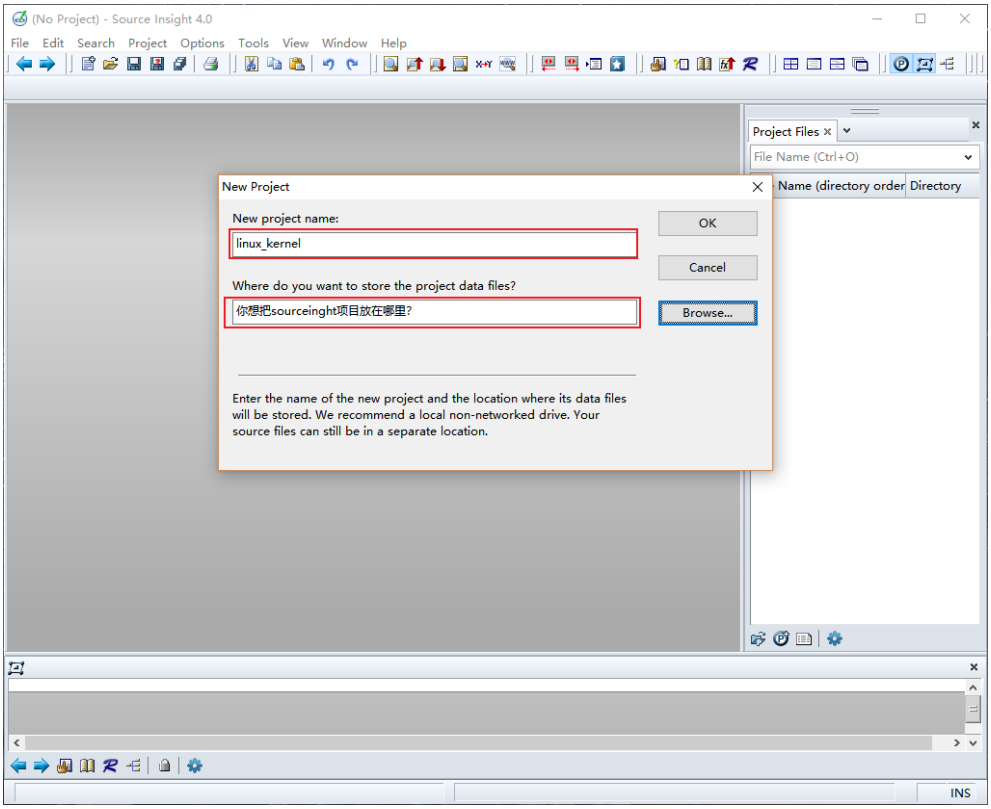


图 2.3.2.2 选择项目文件保存位置

3) 设置 **Project Source Directory – the main location of your source files**: (项目源目录 - 源文件的主要位置:)也就是内核源码所在目录，点击红框左边 … 选择源码目录，点击 OK

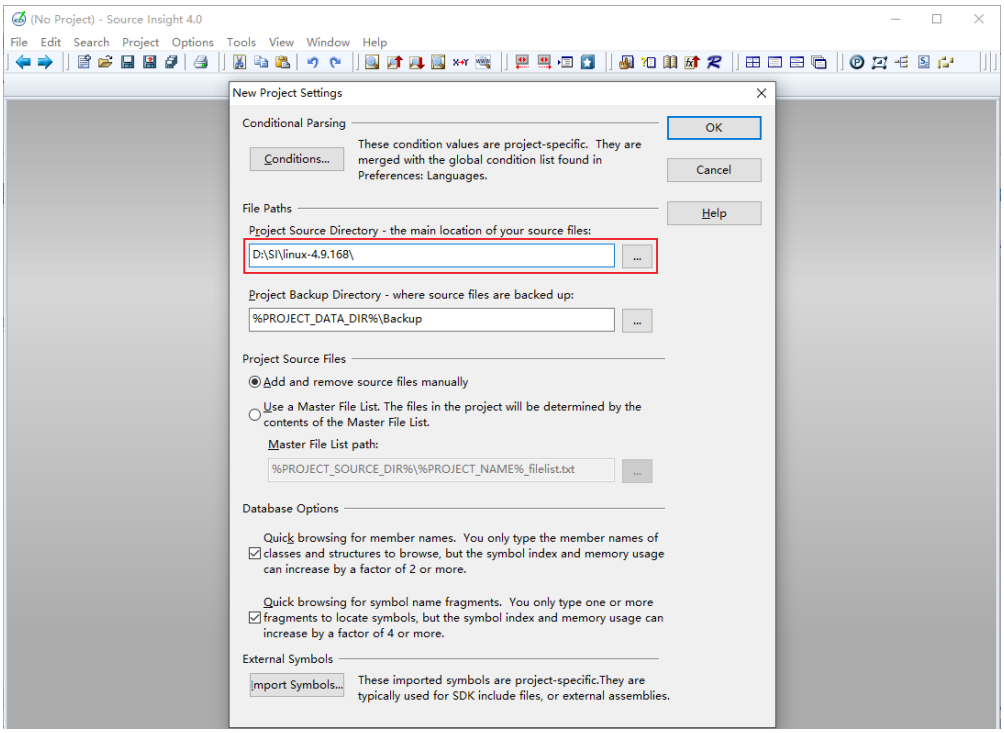


图 2.3.2.3 选择源码所在位置

4) 添加源代码，点击 Add 或 Add All; 其中 Add 是手动选择需要添加的文件，而 Add All 是添加所有

文件，一般我们选择 **Add All**,在弹出的提示框中选中 **Recursively add lower sub-directories**(递归添加下级的子目录)点击 **OK**。同样的 Remove File,Remove All 是移除单个文件或者移除所有文件

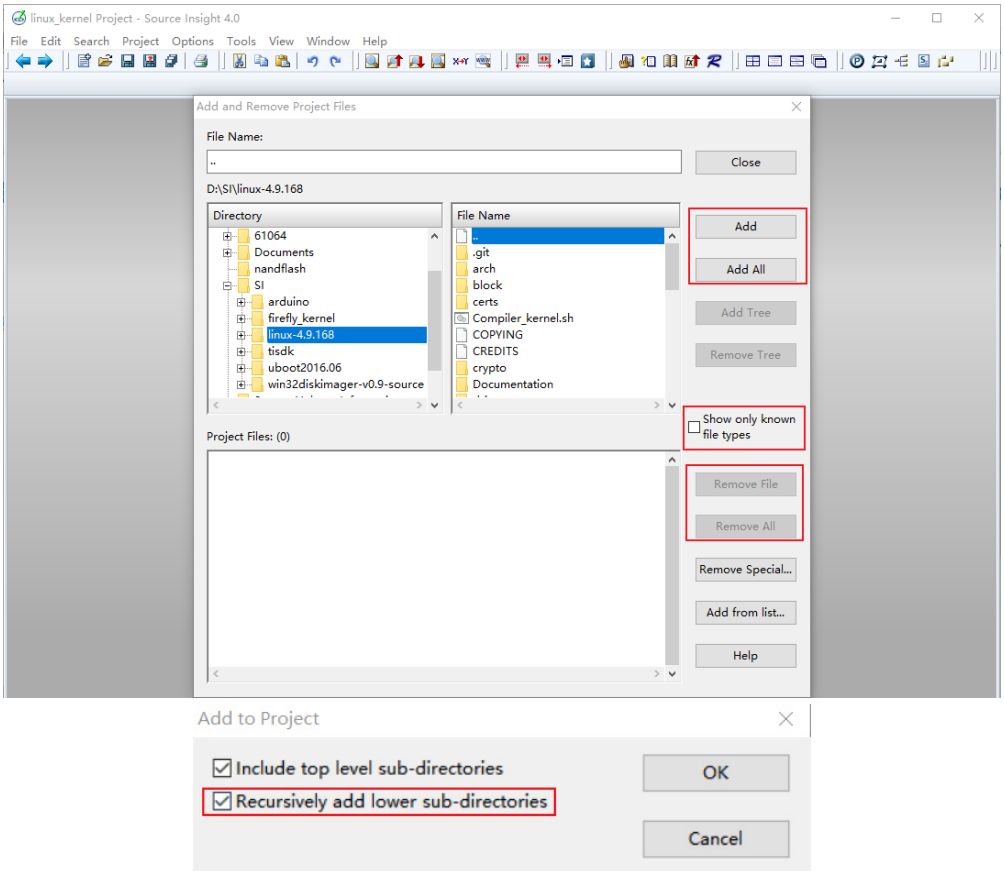


图 2.3.2.4 添加源码文件

5) 添加文件完成后点击 Close，此时界面会返回到主界面

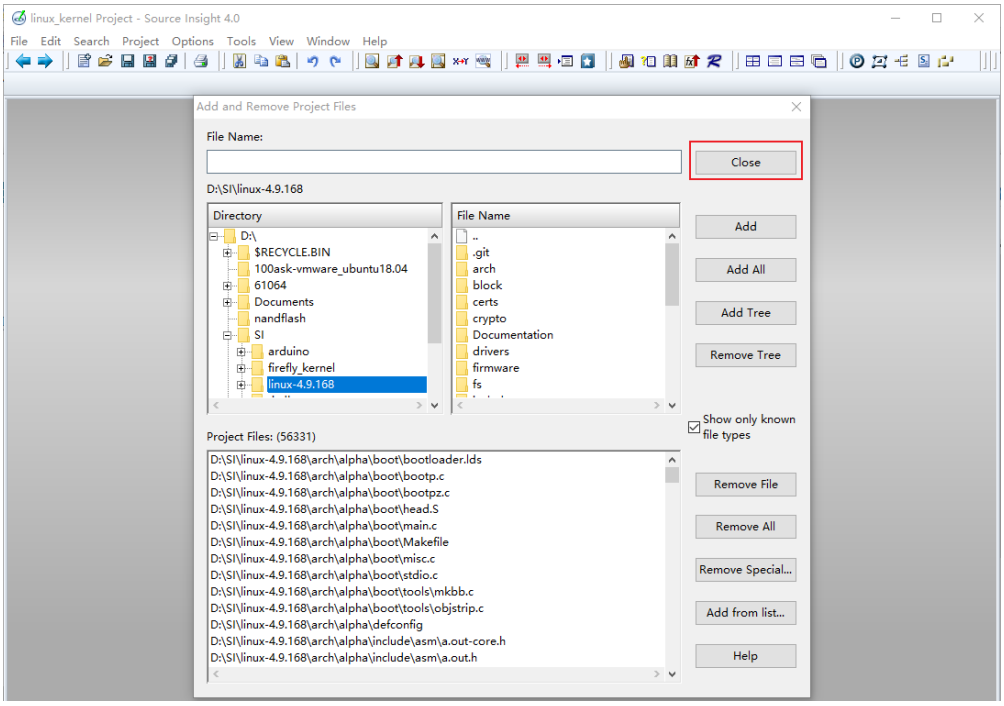


图 2.3.2.5 关闭添加源码窗口

6) 接下来我们需要点击 Synchronize File(同步文件)

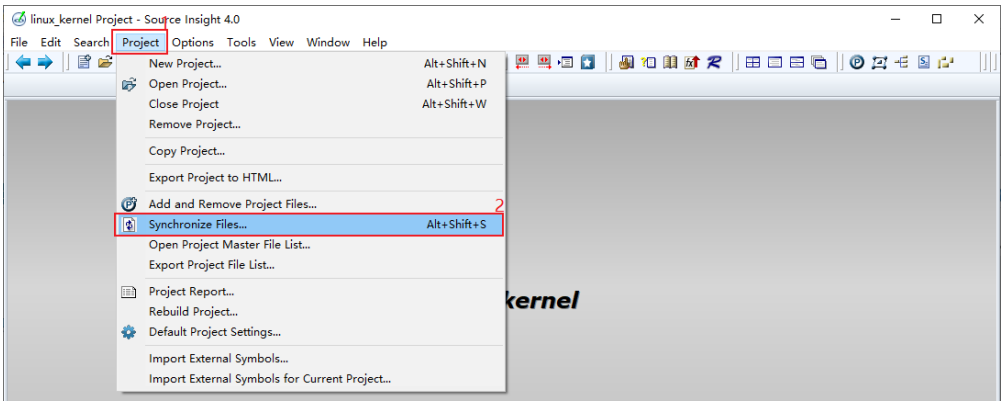


图 2.3.2.6 同步文件

7) 在弹出的 Synchronize File 对话框中 选中 **Force all files to be re-parsed** (强制解析所有文件)，这一步会生成所有代码之间的调用关系等,等待一段时间后会返回到主页面，此时可以进行阅读代码

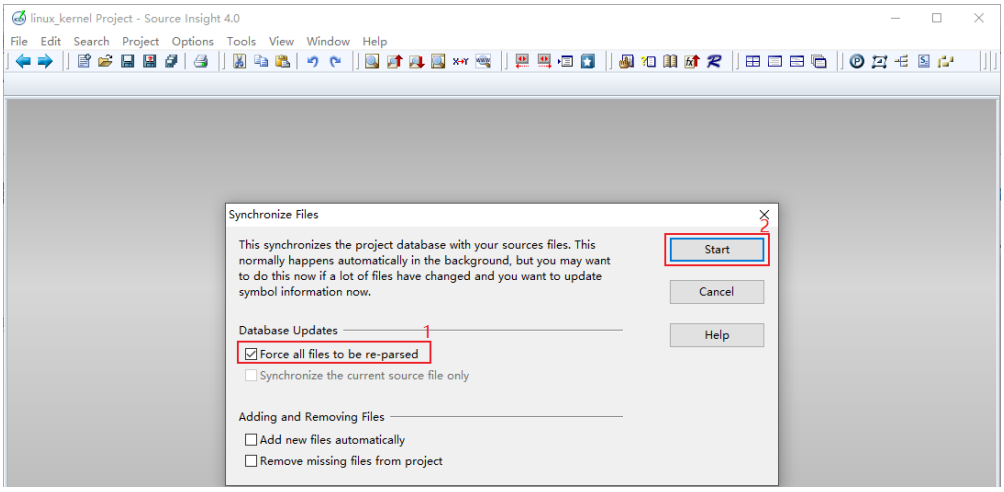


图 2.3.2.7 解析所有文件

2.4.3 Source Insight的操作示例

1) 点击"P"图标打开文件列表，双击文件打开文件

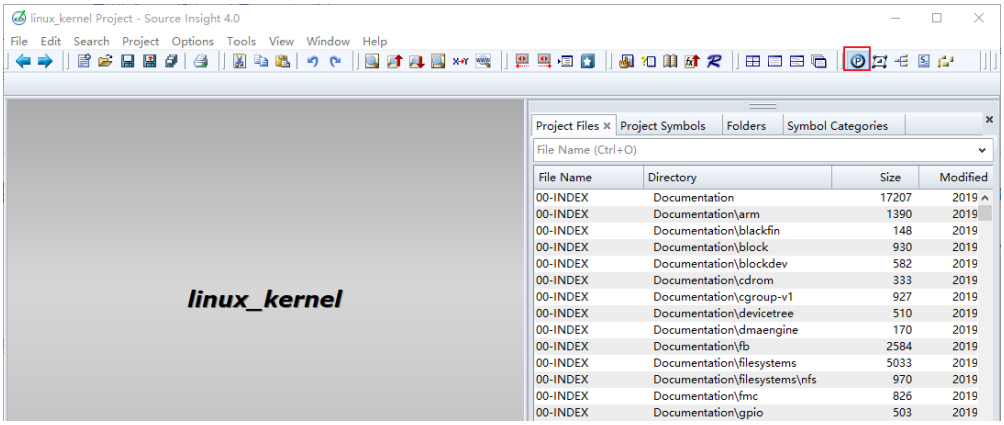


图 2.3.3.1 点击 P 图标

2) 鼠标放在函数、变量上"ctrl+鼠标点击" 跳到定义的位置。

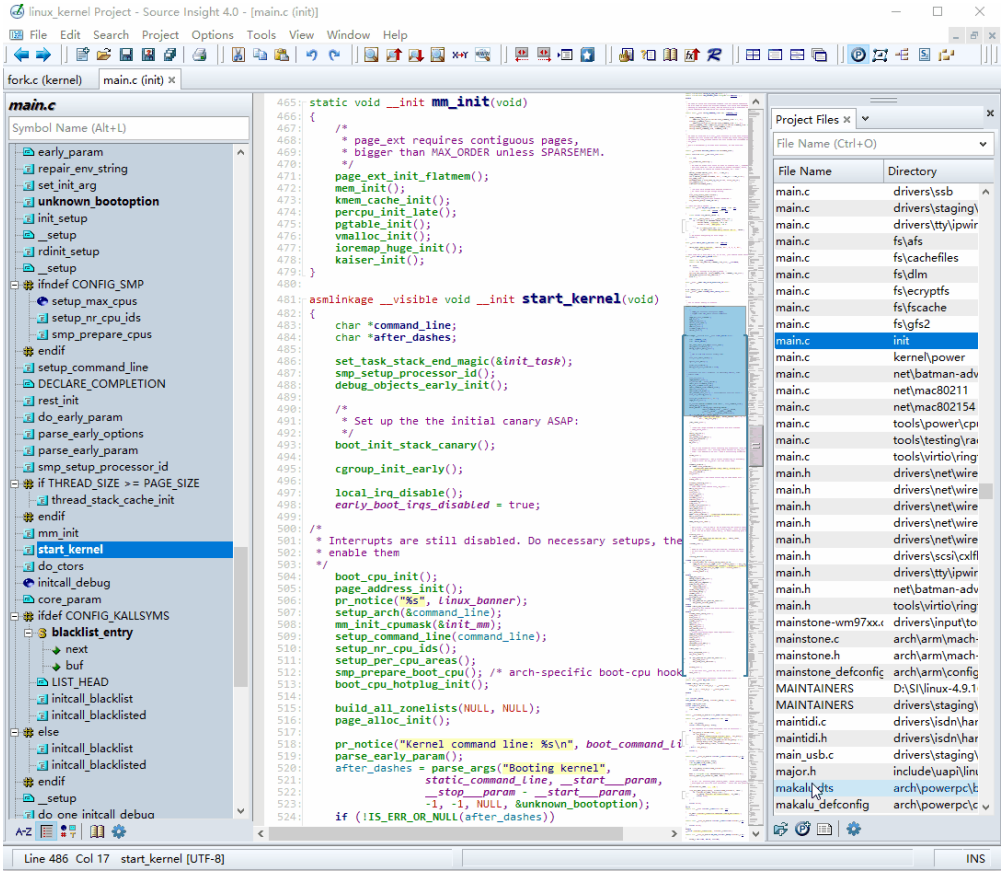


图 2.3.3.2 查找函数或变量定义

3) 双击函数, "ctrl +/"查找引用。

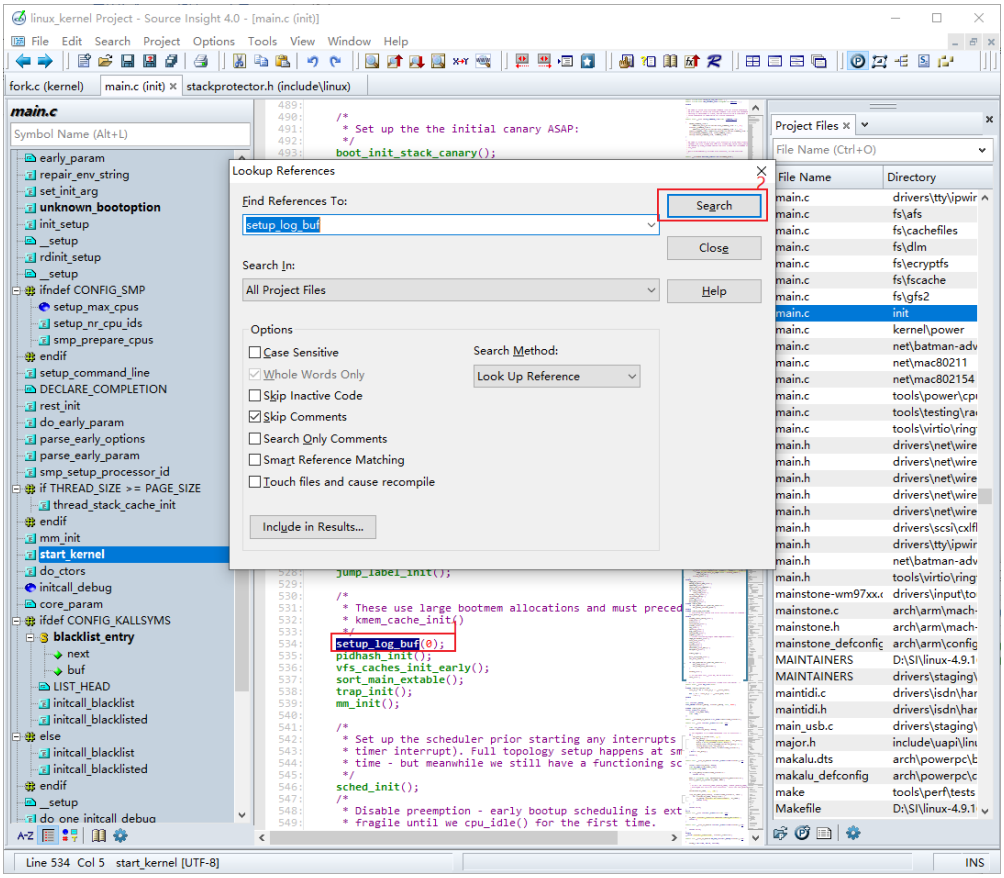


图 2.3.3.3 查找引用

其它一些常用功能键。

- Alt + , 退回上一步操作
- F5 显示行号
- F8 高亮代码或者变量

第3章 构建系统

Linux 平台上有许多开源的嵌入式 linux 系统构建框架(框架的意思就是工具)，这些框架极大的方便了开发者进行嵌入式系统的定制化构建，目前比较常见的有 OpenWrt, Buildroot, Yocto, Arago 等等。其中 Buildroot 功能强大，使用简单，而且采用了类似于 linux kernel 的配置和编译框架，所以受到广大嵌入式开发人员的欢迎。

本章重点介绍使用 Buildroot_2018.02LTS 版构建文件系统和 u-boot, kernel 镜像的方法，并从这三个部分入手，描述如何使用 Buildroot 构建一个适合 AM335X 系列开发板的嵌入式 Linux 系统。

在构建文件系统时，还简要介绍了如何通过 Buildroot 将 QT5 图形系统集成到文件系统中，方便用户后续开发 QT5 的应用程序。

3.1 获取源码

源码的获取方法有 2 种：本地拷贝、在线下载。这 2 种方法请选择 1 种，不要同时选择 2 种方法。

3.1.1 本地拷贝

通过 FileZilla 工具上传资料光盘中的(07_bsp_sdk/100ask_am335x)整个文件夹到 ubuntu 系统/home/book 目录下。

3.1.2 在线下载

100ask_am335x 开发板的所有代码都是保存在 git 站点上通过 repo 命令进行统一管理，可以通过如下命令进行下载或同步。

1) 配置 repo

下载 repo 工具前需要设置 git 的邮箱和用户名，git 邮箱和用户名请根据个人情况进行配置。

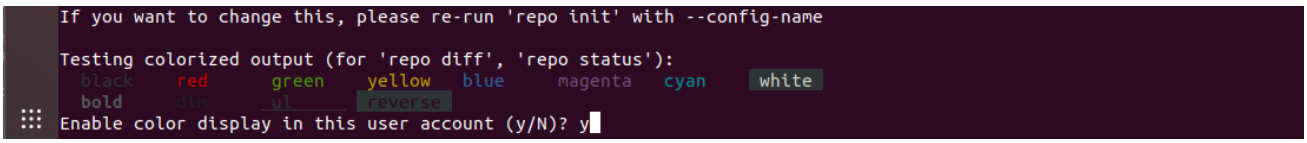
```
book@100ask:~$ git config --global user.email "you@example.com"
book@100ask:~$ git config --global user.name "Your Name"
```

2) 下载源码

通过 repo 管理多个 git 仓库中的源码，可以及时更新最新代码，以方便开发者学习使用。

```
book@100ask:~$ git clone https://git.dev.tencent.com/codebug8/repo.git
book@100ask:~$ mkdir -p 100ask_am335x && cd 100ask_am335x
book@100ask:~/100ask_am335x$ ../repo/repo init -u
https://dev.tencent.com/u/weidongshan/p/manifests/git -b linux-sdk -m ti335x/100ask-
am335x_linux_release_v1.0.xml --no-repo-verify
book@100ask:~/100ask_am335x$    ../repo/repo sync -j4
```

首次下载如果提示 **Testing colorized output (for 'repo diff', 'repo status'):** 此时输入 **y** 即可,继续执行 ../repo/repo sync -j4 命令即可开始同步源码。



同步后源码目录结构如下图所示。



注意：repo 在线下载的代码和资料光盘中的代码是一致的，后期方便同步更新！

3.2 配置交叉编译工具链

注意：使用我们提供的 ubuntu 映象文件时，请按照我们的目录结构，手动设置交叉编译工具链以及编译的架构环境变量配置，（建议配置为永久生效）。

3.2.1 设置交叉编译工具链

交叉编译工具链用来在 ubuntu 主机上编译应用程序，而这些应用程序是在 ARM 等其他平台上运行。

设置交叉编译工具主要是设置 PATH， ARCH 和 CROSS_COMPILE 三个环境变量，下面介绍具体设置方法。

在本文档中，源码、交叉编译工具链都是存放于/home/book 目录下；如果你的目录不一样，请自行修改本节所讲述的命令。

设置这 3 个环境变量有多种方法：

1) 永久生效

如需永久修改，请修改用户配置文件。在 Ubuntu 系统下，修改如下：

```
vim  ~/.bashrc
```

在行尾添加或修改：

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export PATH=$PATH:/home/book/100ask_am335x/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
```

设置完毕后，要重启系统。

2) 临时生效

执行完“export”命令后，该设置**只对当前终端有效：**

```
book@100ask:~$ export PATH=$PATH:/home/book/100ask_am335x/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
book@100ask:~$ export ARCH=arm
book@100ask:~$ export CROSS_COMPILE=arm-linux-gnueabihf-
```

3) 手动指定

先设置 PATH 环境变量，然后在 make 编译时指定 ARCH 架构 CROSS_COMPILE 交叉编译工具链(执行 make 命令时指定的参数，只对当前命令有效；下次执行 make 时仍需要再次指定那些参数)：

```
book@100ask:~$ export PATH=$PATH:/home/book/100ask_am335x/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
book@100ask:~$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

3.2.2 测试交叉编译工具链

测试环境变量：

```
book@100ask:~$ echo $ARCH
arm
book@100ask:~$ echo $CROSS_COMPILE
arm-linux-gnueabihf-
```

测试交叉编译器：

```
book@100ask:~$ arm-linux-gnueabihf-gcc -v
```

```
Using built-in specs.
COLLECT_GCC=arm-linux-gnueabihf-gcc-6.2.1
COLLECT_LTO_WRAPPER=/home/book/100ask_firefly-rk3288/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin/../libexec/gcc/arm-linux-gnueabihf/6.2.1/lto-wrapper
Target: arm-linux-gnueabihf
Configured with: /home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/snapshots/gcc-linaro-6.2-2016.11/configure SHELL=/bin/bash --with-mpc=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/builds/destdir/x86_64-unknown-linux-gnu --with-mpfr=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/builds/destdir/x86_64-unknown-linux-gnu --with-gmp=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/builds/destdir/x86_64-unknown-linux-gnu --with-gnu-as --with-gnu-ld --disable-libstdcxx-pch --disable-libmudflap --with-cloog=no --with-ppl=no --with-isl=no --disable-nls --enable-c99 --enable-gnu-indirect-function --disable-multilib --with-tune=cortex-a9 --with-arch=armv7-a --with-fpu=vfpv3-d16 --with-float=hard --with-mode=thumb --enable-multiarch --with-build-sysroot=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/sysroots/arm-linux-gnueabihf --enable-lto --enable-linker-build-id --enable-long-long --enable-shared --with-sysroot=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/builds/destdir/x86_64-unknown-linux-gnu/arm-linux-gnueabihf/libc --enable-languages=c,c++,fortran,lto --enable-checking=release --disable-bootstrap --build=x86_64-unknown-linux-gnu --host=x86_64-unknown-linux-gnu --target=arm-linux-gnueabihf --prefix=/home/tcwg-buildslave/workspace/tcwg-make-release/label/docker-trusty-amd64-tcwg-build/target/arm-linux-gnueabihf/_build/builds/destdir/x86_64-unknown-linux-gnu
Thread model: posix
gcc version 6.2.1 20161016 (Linaro GCC 6.2-2016.11)
```

3.3 解压编译 bootloader

我们使用的版本为 u-boot-2016.06,我们提供的源码针对板子进行过修改，u-boot 官网下载的源码不能直接使用。

3.3.1 解压u-boot源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 U-boot 源码：

```
book@100ask:~$ cd ~/100ask_am335x
book@100ask: ~/100ask_am335x$ tar -zxvf u-boot2016.06.tar.gz
```

3.3.2 编译u-boot镜像

不同的开发板对应不同的配置文件，配置文件位于 u-boot 源码的 configs/ 目录,下面以 100ask_am335x 开发板为例，说明 u-boot 的编译过程：

```
book@100ask: ~/100ask_am335x$ cd u-boot2016.06
book@100ask:~/100ask_am335x/uboot2016.06$ make distclean
book@100ask:~/100ask_am335x/uboot2016.06$ make 100ask_am335x_defconfig
book@100ask:~/100ask_am335x/uboot2016.06$ make
```

编译完成之后生成 MLO 和 u-boot.img，可以用于 TF 卡启动和 EMMC 启动。为方便学习，建议优先选择从 TF 卡启动。

烧写 MLO 文件以及 u-boot.img 文件方式请参考 Nandflash 启动/更新系统

在 U-Boot 启动过程中，通过按空格键可以进入 U-Boot 命令控制台，然后执行各种命令。例如：

```
>#help -- 获取帮助
>#echo -- 查看变量
```

注明： 如何设置启动参数请参考 NFS ROOT 启动

3.4 编译 linux kernel

3.4.1 解压kernel源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 Kernel 源码：

```
book@100ask:~$ cd ~/100ask_am335x/
book@100ask:~/100ask_am335x$ tar -zxvf linux-4.9.168.tar.gz
```

3.4.2 编译内核镜像

不同的开发板对应不同的配置文件，配置文件位于内核源码 arch/arm/configs/目录。下面以 100ask_am335x 开发板为例，说明 kernel 的编译过程：

```
book@100ask:~/100ask_am335x$ cd linux-4.9.168
book@100ask:~/100ask_am335x/linux-4.9.168$ make mrproper
book@100ask:~/100ask_am335x/linux-4.9.168$ make 100ask_am335x_defconfig
book@100ask:~/100ask_am335x/linux-4.9.168$ make zImage
book@100ask:~/100ask_am335x/linux-4.9.168$ make dtbs
```

编译完成后，在 arch/arm/boot 目录下生成 zImage 内核文件，在 arch/arm/boot/dts 目录下生成设备树的二进制文件 100ask-am335x.dtb。

3.4.3 编译内核模块并安装

编译内核模块

```
book@100ask:~/100ask_am335x/linux-4.9.168$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-  modules
```

安装内核模块到 nfs 根文件系统

```
book@100ask:~/100ask_am335x/linux-4.9.168$ sudo make  ARCH=arm INSTALL_MOD_PATH=/media/rootfs  modules_install
```

3.5 构建根文件系统

3.5.1 解压buildroot源码

注意：如果你使用的是 repo 在线同步代码，则不需要下面的解压步骤。

对于本地拷贝，执行下面的命令解压 buildroot 源码：

```
book@100ask:~$ cd ~/100ask_am335x
book@100ask:~/100ask_am335x$ tar -zxvf buildroot2018.02.tar.gz
```

3.5.2 配置文件说明

配置文件	含义
100ask_am335x _defconfig	默认文件系统版本(这个版本不支持 QT)
100ask_am335x_qt_defconfig	包含 Qt 文件系统版本

3.5.3 编译系统

下面以 100ask_am335x_defconfig 配置文件为例，说明 Buildroot 的配置过程：

```
book@100ask:~/100ask_am335x$ cd buildroot2018.02
book@100ask:~/100ask_am335x/buildroot2018.02$ make clean
book@100ask:~/100ask_am335x/buildroot2018.02$ make 100ask_am335x_defconfig
book@100ask:~/100ask_am335x/buildroot2018.02$ make menuconfig
book@100ask:~/100ask_am335x/buildroot2018.02$ make –jN              /* N表示CPU的线程数，比如可以写4、8等以加快编译速度 */
```

注意：机器性能不同，编译时间不同。性能差的电脑，有可能需要等待 1 ~ 2 个小时。

3.5.4 镜像文件

编译成功后文件输出路径为 **output/images**

```
buildroot2018.02
        ├── output
            ├── images
```

```
├──── 100ask-am335x.dtb          <--设备树文件
├──── boot.vfat                  <--分区文件
├──── MLO                        <--启动引导
├──── ramdisk.gz                 <--RAM格式根文件系统
├──── rootfs.cpio
├──── rootfs.cpio.gz
├──── rootfs.cpio.uboot
├──── rootfs.ext2                <--ext2格式根文件系统
├──── rootfs.ext4 -> rootfs.ext2
├──── rootfs.tar                 <--打包后的根文件系统
├──── rootfs.tar.gz              <--打包并压缩的根文件系统，用于NFSROOT启动
├──── rootfs.ubi                 <--支持nandflash加载的根文件系统
├──── rootfs.ubifs
├──── sdcard.img                 <--完整的SD卡系统镜像
├──── u-boot.bin                 <--qspi启动的u-boot
├──── u-boot.img                 <--u-boot镜像
├──── u-boot-spl.bin             <--spl启动镜像
├──── uEnv.txt                   <--默认u-boot配置文件
└──── zImage                     <--内核镜像
```

注明：如何通过 NFS 系统根文件系统进行调试，请参考 NFS ROOT 启动章节

第4章 烧写系统

本章主要介绍 100ask_am335x 的三种启动方式，分别是：

① 板载 nandflsh 启动

② TF 卡启动

③ nfs 网络挂载方式启动。

其中可用于 TF 卡启动的系统有两个，一个是基于 buildroot 构建的根文件系统，另一个是基于 arm 架构的 Ubuntu 发行版系统，用户可以根据自己的需要选择适合自己的进行安装。

本章需要使用的软件：

- HP USB Disk Storage Format Tool 2.0.6.EXE
- win32diskimager-1.0.0-install.exe

100ask_am335x 开发板支持 SD 卡、Nand Flash 两种启动方式。使用 Nand Flash 启动之前，如果 Nand Flash 上没有可用的系统，则要借助 SD 卡把系统烧写到 Nand Flash 上。

SD 卡启动、Nand Flash 启动，是二选一，只能选择一个。

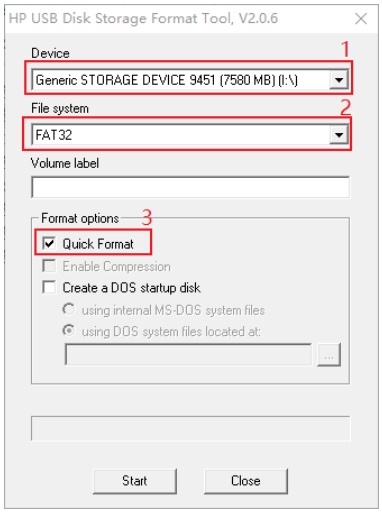
4.1 Nand Flash 启动/更新系统

4.1.1 格式化SD卡

进入资料光盘 01_tools 目录，右键以管理员权限运行 HP USB Disk Storage Format Tool 2.0.6.EXE 程序，来格式化 SD 卡，如下图操作：

1) 选择需要格式化的 sd 卡
2) 选择 sd 卡文件系统类型为 FAT32
3) 选中 Quick Format(快速格式化)
4) 点击 start 开始格式化

注意： 如果点击 start 时提示错误，则在错误信息框中点击确定，然后再次点击 start 就可以了。



4.1.2 更新系统

1) 拷贝文件：

拷贝资料光盘中（\02_images\nandflash_system）目录下所有文件到格式化好的 SD 卡中，如下图所

示：

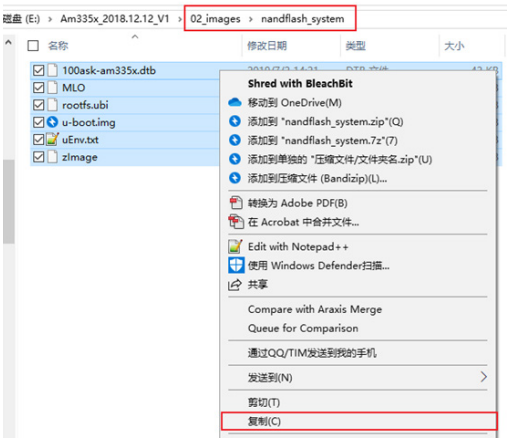


图 4.1.2.1 拷贝 nandflash_system 镜像文件

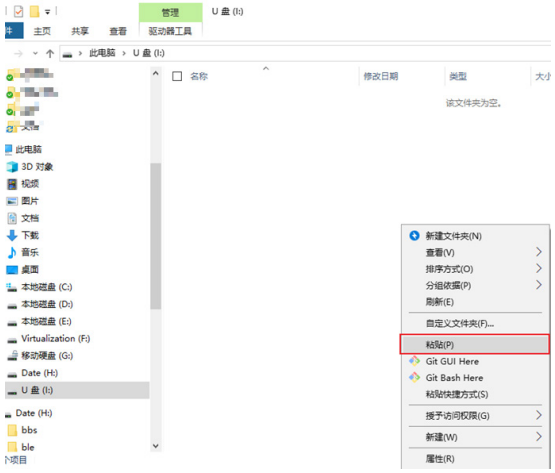
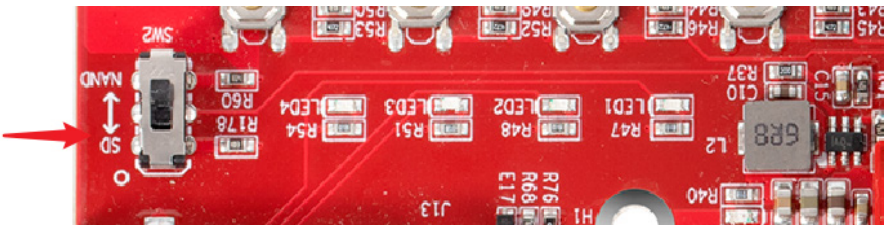


图 4.1.2.2 拷贝 nandflash 到格式化的 SD 卡中

2) 使用 SD 卡启动板子：

把 SD 卡插到开发板上，并设置启动方式为 mmc 启动（把启动开关拨到如下图 SD 位置）：



接好串口线后，打开串口终端，开发板上电，串口有输出时按下空格键即可进入 u-boot 参数设置界面，如下所示：

```
CCCCCCCC
U-Boot SPL 2016.05-gddf5183-dirty (Jul 07 2019 - 21:40:30)
PMIC detected:TPS65217
Trying to boot from MMC1
reading u-boot.img
reading u-boot.img


U-Boot 2016.05-gddf5183-dirty (Jul 07 2019 - 21:40:30 -0400)

CPU  : AM335X-GP rev 2.1
       Watchdog enabled
I2C:   ready
DRAM:  512 MiB
NAND:  512 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
*** Warning - bad CRC, using default environment

Net:   cpsw
Press SPACE to abort autoboot in 2 seconds
100ask#
```

3) 使用 U-Boot 更新 Nand Flash：

在 u-boot 界面输入“run updatesys”并回车，这就开始把 SD 卡中的系统文件烧写到 Nand Flash 上去。

```
100ask# run updatesys
```

在烧写过程中，打印“Writing at ”，会打印烧写进度，直到进度为 “100% is complete”，表示烧写完成。

烧写过程全部结束时，会打印“**nand flash system update OK!** ”。

此时可以关闭开发板电源，拔出 SD 卡,设置启动方式为 Nand Flash 启动，开发板重新上电即可！

烧写过程的输出信息如下：

```
100ask# run updatesys

NAND erase.chip: device 0 whole chip
Erasing at 0x1ffe0000 -- 100% complete.
OK
switch to partitions #0, OK
mmc0 is current device
reading MLO
52824 bytes read in 7 ms (7.2 MiB/s)
HW ECC BCH8 Selected

NAND write: device 0 offset 0x0, size 0xce58
Writing at 0x0 -- 100% is complete. 52824 bytes written: OK
reading 100ask-am335x.dtb
42526 bytes read in 8 ms (5.1 MiB/s)
HW ECC BCH8 Selected

NAND write: device 0 offset 0x80000, size 0xa61e
Writing at 0x80000 -- 100% is complete. 42526 bytes written: OK
reading u-boot.img
433640 bytes read in 28 ms (14.8 MiB/s)
HW ECC BCH8 Selected

NAND write: device 0 offset 0xc0000, size 0x69de8
Writing at 0x120000 -- 100% is complete. 433640 bytes written: OK
reading zImage
5274608 bytes read in 292 ms (17.2 MiB/s)
HW ECC BCH8 Selected

NAND write: device 0 offset 0x200000, size 0x507bf0
Writing at 0x700000 -- 100% is complete. 5274608 bytes written: OK
reading rootfs.ubi
139067392 bytes read in 7503 ms (17.7 MiB/s)
SW ECC selected

NAND write: device 0 offset 0xa00000, size 0x84a0000
Writing at 0x8e80000 -- 100% is complete. 139067392 bytes written: OK
nand flash system update OK!
100ask#
```

4.2 SD 卡启动

4.2.1 安装映象烧写工具win32diskimager

安装资料光盘下 **01_tools/ win32diskimager-1.0.0-install.exe** 软件，以后运行它时要“以管理员身份运行”。

4.2.2 烧写映像文件

1) 把 SD 卡通过读卡器插入电脑
2) 使用 win32diskimager 烧写映像文件 sdcard.img

“以管理员身份运行”win32diskimager，如下图选择 SD 卡、选择映像文件 sdcard.img，然后点击“写入”,操作步骤如下图 4.2.2.1 所示：

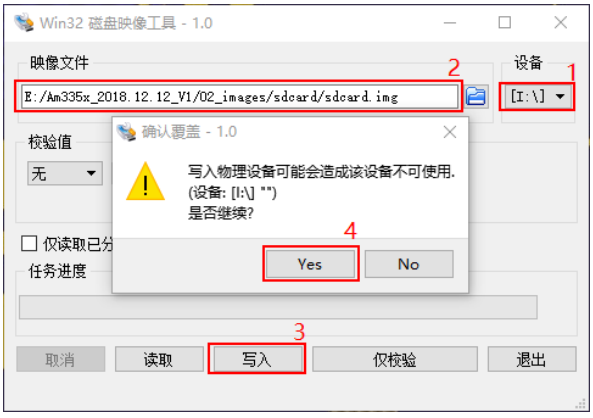


图 4.2.2.1 镜像文件烧写示意图

上图中各序号含义为：

1. 选择 SD 卡

2. 选择映象文件 sdcard.img
3. 点击“写入”
4. 确定要写入。

烧写成功后，弹出对话框“**写入成功**”，此时点击 OK，拔下 SD 卡，将启动开关拨到 SD 卡处，上电启动开发板。

4.3 NFS ROOT 启动

Buildroot 编译完成之后生成的 rootfs.tar.gz，可以解压之后放到 NFS 服务器上作为 NFS ROOT 文件系统供开发板使用。使用 NFS 文件系统，便于程序的开发调试。所谓 NFS 服务器，就是我们在 VMWare 上运行的 Ubuntu。

使用 NFS 文件系统时，我们一般还会在 u-boot 使用 tftpboot 命令从 ubuntu 中下载内核文件 zImage，所以 ubuntu 上既要配置 NFS 服务，也要配置 TFTP 服务。

注意：使用我们提供的 ubuntu 映象文件时，本节内容中所描述的配置工作都已经实现，不再需要阅读 4.3.1、4.3.2 这两节的内容。

如果你使用的不是我们提供的 ubuntu 映象文件，可以参考这两节来配置网络环境。

4.3.1 安装TFTP服务端：

在 ubuntu 中执行以下命令安装 TFTP 服务：

```
book@100ask:~$ sudo apt-get install tftp-hpa tftpd-hpa
```

然后，创建 TFTP 服务器工作目录,并打开 TFTP 服务配置文件,如下:

```
book@100ask:~$ mkdir -p /home/book/tftpboot
book@100ask:~$ chmod 777 /home/book/tftpboot
book@100ask:~$ sudo vim /etc/default/tftpd-hpa
```

在配置文件/etc/default/tftpd-hpa 中，添加以下字段：

```
TFTP_DIRECTORY="/home/book/tftpboot"
TFTP_OPTIONS="-l -c -s"
```

最后，重启 TFTP 服务:

```
book@100ask:~$ sudo service tftpd-hpa restart
```

4.3.2 安装NFS服务：

NFS 即网络文件系统，允许开发板直接通过网络挂载 PC 机中的文件夹。下面介绍在 ubuntu 上的 NFS 服务安装和配置。

首先，执行以下命令安装 NFS 服务：

```
book@100ask:~$ sudo apt-get install nfs-kernel-server
```

然后编辑/etc/exports 文件，添加 NFS 服务导出的工作目录：

```
book@100ask:~$ sudo vim /etc/exports
```

添加 NFS 目录：下面以/home/book/rootfs 为例，将其添加到/etc/exports 文件中，如下：

```
/home/book/nfs_rootfs   *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)
```

最后，重启 NFS 服务:

```
book@100ask:~$ sudo service nfs-kernel-server restart
```

如果一切正常，可以在 ubuntu 中测试 NFS 服务：

```
book@100ask:~$ sudo   mount   -t   nfs   127.0.0.1:/home/book/nfs_rootfs      /mnt
```

在开发板上，可以运行这样的命令挂载NFS：mount -t nfs -o nolock,vers=3 xxx.xxx.xxx.xxx:/home/book/nfs_rootfs /mnt
假设Ubuntu的IP是xxx.xxx.xxx.xxx

4.3.3 在ubuntu中准备好文件

可以在 U-Boot 中通过 tftpboot 命令从 ubuntu 中下载内核文件 zImage 100ask_am335x.dtb，可以设置内核的启动参数让单板启动后通过 NFS 挂载 ubuntu 中的某个目录作为根文件系统。

注意：所谓根文件系统就是类似 Windows 的 C 盘，里面存放有必须的 APP、库文件、配置文件。通过 NFS 可以把 Ubuntu 的某个目录，当作板子的“C 盘”——Linux 中称之为根文件系统。

在操作单板之前，需要在 ubuntu 上准备好这些内容：zImage、100ask_am335x.dtb、nfs_rootfs 文件夹。

1) 拷贝内核和设备树文件到 tftp 目录：

将出厂镜像或者自行编译的 zImage 和设备树文件 100ask-am335x.dtb，拷贝到 ubuntu 的/home/book/tftpboot 目录。

2) 把根文件系统压缩包解压到 NFS 目录：

把使用 buildroot 构建得到的根文件系统 nfs_rootfs/rootfs.tar.gz，复制、解压到 ubuntu 的/etc/exports 文件中指定的目录里，即复制到/home/book 目录下，得到/home/book/nfs_rootfs 下众多文件：

```
book@100ask:~/100ask_am335x/buildroot2018.02/output/images$ cp -rf nfs_rootfs/   ~
book@100ask:~/100ask_am335x/buildroot2018.02/output/images$ cd ~/nfs_rootfs
book@100ask:~/nfs_rootfs$ sudo tar -zxvf rootfs.tar.gz
```

4.3.4 在u-boot中通过网络启动单板

既然要使用网络启动单板，那么双方(开发板、ubuntu)就要确保网络是联通的，假设 ubuntu 的 IP 为 192.168.1.132，开发板的 IP 为 192.168.1.112。

首先，在 U-boot 中设置开发板 IP 为 192.168.1.112, 如下:

```
># setenv ipaddr 192.168.1.112
```

然后，在 U-boot 中使用 ping 命令测试开发板与 NFS 服务器是否连通(出现“alive”就表示联通):

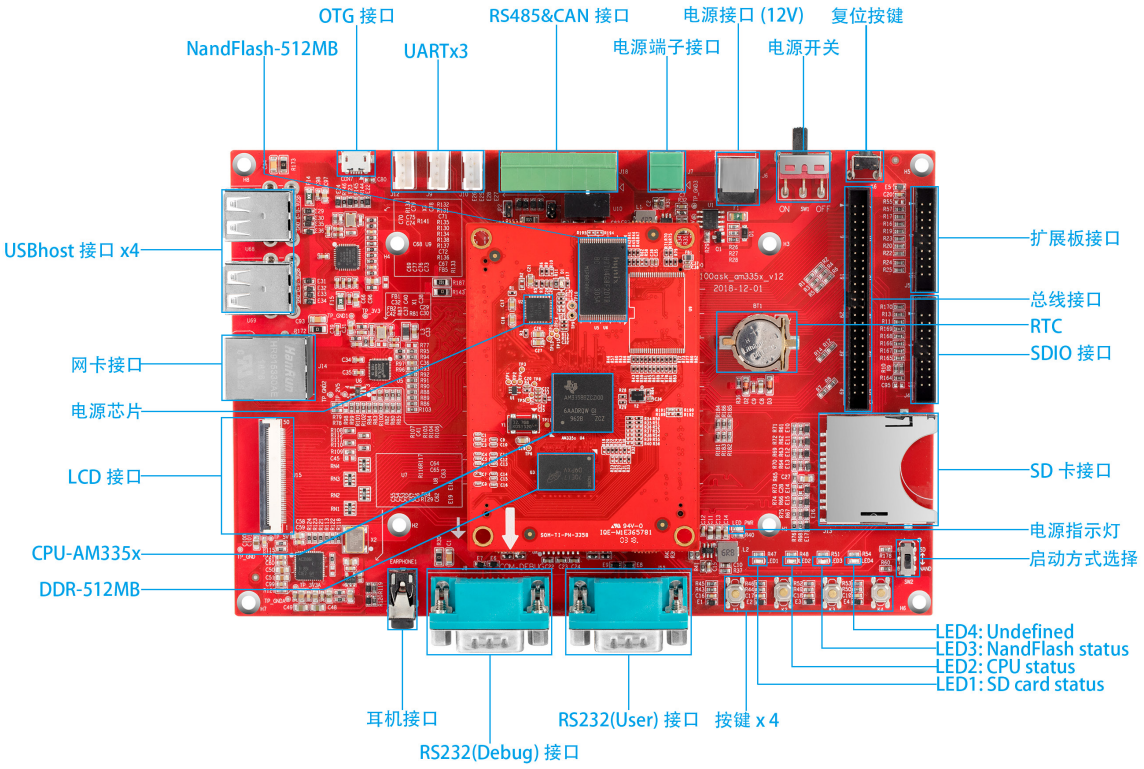
```
# ping 192.168.1.111
```

最后，在 U-Boot 控制台执行以下命令启动单板:

```
># setenv serverip 192.168.1.132
># setenv ipaddr 192.168.1.112
># setenv fdtfile 100ask-am335x.dtb
># setenv rootpath /home/book/rootfs,vers=3
># run netboot
```

第5章平台功能测试

本章介绍如何测试板载功能，开发板各个功能部件所在位置以及名称如下图所示。本章所有的功能测试均在串口登录终端下进行。



5.1 网卡接口测试

此节演示在串口终端下如何设置开发板的 ip 地址，测试网络的连通性。并使用电脑通过网络浏览器访问开发板，验证开发板的 http+php 可用。

注意：既然是在开发板和电脑之间测试网络，那双方需要有网络连接。两者之间需要有一个路由器，开发板通过网线与路由器连接。而电脑与路由器之间，可以使用网线连接，也可以使用 WIFI 连接。

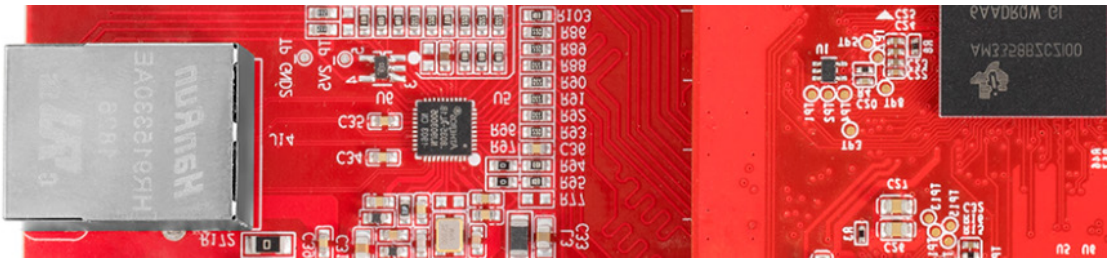


图 5.1.1 网卡接口位置示意图

1) 通过 ifconfig 命令查看 ip 地址：

```
[root@100ask_am335x:~]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.87  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::4389:aa2c:b99e:cbfb  prefixlen 64  scopeid 0x20<link>
        ether 98:5d:ad:42:a9:74  txqueuelen 1000  (Ethernet)
        RX packets 32  bytes 4038 (3.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 16  bytes 1520 (1.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 176

[root@100ask_am335x:~]#
```

通过上图可知，开发板已经自动获得 IP 地址 192.168.1.87(你的开发板自动获取的 IP 可能不一样)。

如果开发板未能获取 IP，则可以使用 **udhcpc** 命令再次尝试获取 IP，如下图所示(“192.168.1.87 obtained”表示获取到了 IP 192.168.1.87)：

```
[root@100ask_am335x:~]# udhcpc
udhcpc: started, v1.27.2
udhcpc: sending discover
udhcpc: sending select for 192.168.1.87
udhcpc: lease of 192.168.1.87 obtained, lease time 172800
deleting routers
adding dns 192.168.1.1
adding dns 192.168.1.1
[root@100ask_am335x:~]#
```

如果通过 udhcpc 命令无法获得 IP，也可以使用 ifconfig 命令强制设置 IP：

如下图使用 ifconfig 命令强制指定 IP 地址为 192.168.1.17

```
[root@100ask_am335x:~]# ifconfig  eth0 192.168.1.17
[root@100ask_am335x:~]# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.17  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::3631:982d:aee4:edac  prefixlen 64  scopeid 0x20<link>
        ether ec:24:b8:78:9d:d7  txqueuelen 1000  (Ethernet)
        RX packets 17  bytes 3301 (3.2 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 26  bytes 3739 (3.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 176
```

2) 网络连通性测试：

在开发板上执行如下命令，如果有数据返回则表示开发板跟互联网是连通的(前提是你的路由器是可以上网的)：

```
[root@100ask_am335x:~]# ping www.baidu.com
[root@100ask_am335x:~]# ping wwww.baidu.com
PING wwww.baidu.com (220.181.38.251): 56 data bytes
64 bytes from 220.181.38.251: seq=0 ttl=53 time=37.928 ms
64 bytes from 220.181.38.251: seq=1 ttl=53 time=37.749 ms
64 bytes from 220.181.38.251: seq=2 ttl=53 time=37.768 ms
64 bytes from 220.181.38.251: seq=3 ttl=53 time=37.744 ms
^C
--- wwww.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 37.744/37.797/37.928 ms
[root@100ask_am335x:~]#
```

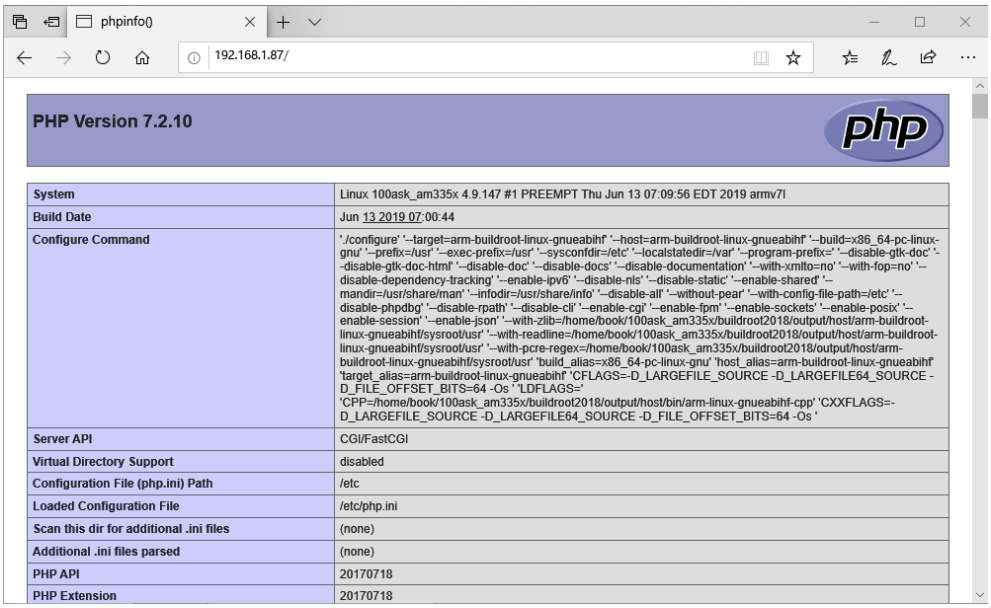
当然，很多时候开发板不能正通互联网，这也没关系，只要能 ping 通我们的 Windows、Ubuntu 即可：

比如我们的 Ubuntu 系统 IP 地址为 192.168.1.11 此时可以通过 ping 命令测试两者是否可以相互通信

```
[root@100ask_am335x:~]# ping 192.168.1.11
PING 192.168.1.11 (192.168.1.11): 56 data bytes
64 bytes from 192.168.1.11: seq=0 ttl=64 time=1.440 ms
64 bytes from 192.168.1.11: seq=1 ttl=64 time=0.886 ms
64 bytes from 192.168.1.11: seq=2 ttl=64 time=0.455 ms
64 bytes from 192.168.1.11: seq=3 ttl=64 time=0.512 ms
^C
--- 192.168.1.11 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.455/0.823/1.440 ms
[root@100ask_am335x:~]#
```

3) http+Php 测试：

假设开发板的 IP 为 **192.168.1.87**，在电脑的网络浏览器中输入此 IP，就能看到此开发板所支持的 php 特征描述：



System	Linux 100ask_am335x 4.9.147 #1 PREEMPT Thu Jun 13 07:09:56 EDT 2019 armv7l
Build Date	Jun 13 2019 07:00:44
Configure Command	'./configure' '--target=arm-buildroot-linux-gnueabihf' '--host=arm-buildroot-linux-gnueabihf' '--build=x86_64-pc-linux-gnu' '--prefix=/usr' '--exec-prefix=/usr' '--sysconfdir=/etc' '--localstatedir=/var' '--program-prefix=' '--disable-gtk-doc' '--disable-gtk-doc-html' '--disable-doc' '--disable-docs' '--disable-documentation' '--with-xmlto=no' '--with-fop=no' '--disable-dependency-tracking' '--enable-ipv6' '--disable-nls' '--disable-static' '--enable-shared' '--mandir=/usr/share/man' '--infodir=/usr/share/info' '--disable-all' '--without-pear' '--with-config-file-path=/etc' '--disable-phpdbg' '--disable-rpath' '--disable-cli' '--enable-cgi' '--enable-fpm' '--enable-sockets' '--enable-posix' '--enable-session' '--enable-json' '--with-zlib=/home/book/100ask_am335x/buildroot2018/output/host/arm-buildroot-linux-gnueabihf/sysroot/usr' '--with-readline=/home/book/100ask_am335x/buildroot2018/output/host/arm-buildroot-linux-gnueabihf/sysroot/usr' '--with-pcre-regex=/home/book/100ask_am335x/buildroot2018/output/host/arm-buildroot-linux-gnueabihf/sysroot/usr' 'build_alias=x86_64-pc-linux-gnu' 'host_alias=arm-buildroot-linux-gnueabihf' 'target_alias=arm-buildroot-linux-gnueabihf' 'CFLAGS=-D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os ' 'LDFLAGS=' 'CPP=/home/book/100ask_am335x/buildroot2018/output/host/bin/arm-linux-gnueabihf-cpp' 'CXXFLAGS=-D_LARGEFILE_SOURCE -D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64 -Os '
Server API	CGI/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	(none)
Additional .ini files parsed	(none)
PHP API	20170718
PHP Extension	20170718

5.2 USB Host 接口测试

此节演示在终端下如何在 USB Host 接口上使用 usb 存储设备。

注意：需要准备一个 USB 设备，比如 U 盘、USB 蓝牙模块、usb 网卡或者 usb 摄像头等。

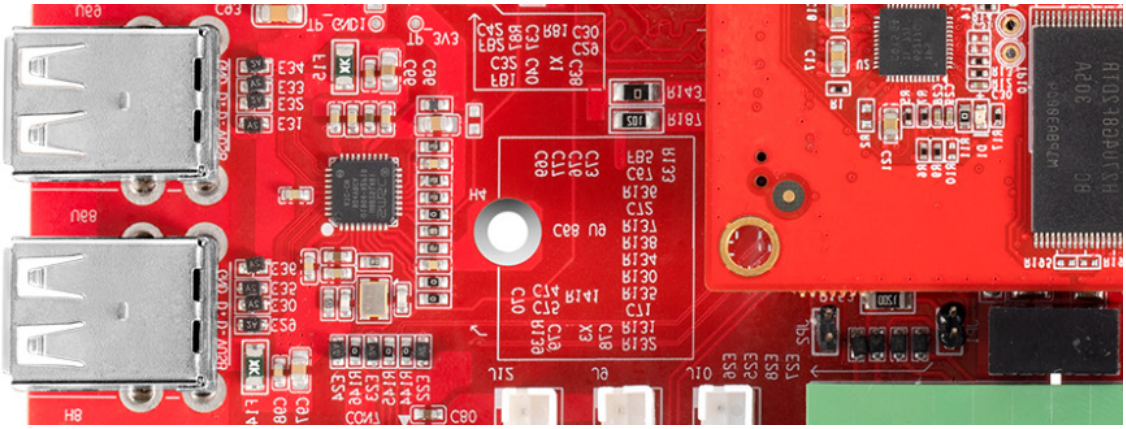


图 5.1.1USB host 接口位置示意图

下面使用一个 16G 的 U 盘作为例子，插到任意一个 USB Host 接口，会打印出如下设备信息：

```
[root@100ask_am335x:~]# [   30.953992] usb 2-1.3: new high-speed USB device number 3 using musb-hdrc
[   31.085002] usb 2-1.3: New USB device found, idVendor=13fe, idProduct=5500
[   31.092195] usb 2-1.3: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[   31.100242] usb 2-1.3: Product: V60
[   31.103902] usb 2-1.3: Manufacturer: EAGET
[   31.108431] usb 2-1.3: SerialNumber: 070D5CA1380C8F92
[   31.118174] usb-storage 2-1.3:1.0: USB Mass Storage device detected
[   31.134978] scsi host0: usb-storage 2-1.3:1.0
[   32.165416] scsi 0:0:0:0: Direct-Access     EAGET    V60              PMAP PQ: 0 ANSI: 6
[   32.183586] sd 0:0:0:0: [sda] 30302208 512-byte logical blocks: (15.5 GB/14.4 GiB)
[   32.194810] sd 0:0:0:0: [sda] Write Protect is off
[   32.200187] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[   32.354321]  sda: sda1 sda2
[   32.369513] sd 0:0:0:0: [sda] Attached SCSI removable disk
[   32.571887] random: crng init done

[root@100ask_am335x:~]#
```

通过打印的设备信息可知，系统为该 usb 存储设备创建的设备节点为 /dev/sda。一般来说/dev/sda 对应整个 U 盘，/dev/sda1 对应该 U 盘的第 1 个分区，/dev/sda2 对应第 2 个分区。

有些 U 盘没有划分分区，它只有一个设备节点/dev/sda，而没有/dev/sda1 等节点。对于这种情况，/dev/sda 既代表整个 U 盘，也代表第 1 个分区。

我们可以挂载某个分区，挂载之前要先通过 fdisk 命令获取分区类型,如下所示：

```
[root@100ask_am335x:~]# fdisk -l /dev/sda
Disk /dev/sda: 14 GB, 15514730496 bytes, 30302208 sectors
1886 cylinders, 255 heads, 63 sectors/track
Units: cylinders of 16065 * 512 = 8225280 bytes

Device  Boot StartCHS    EndCHS        StartLBA     EndLBA    Sectors  Size Id Type
/dev/sda1 *  33,37,5     1023,254,63     532480   29786111   29253632 13.9G  7 HPFS/NTFS
/dev/sda2    1023,254,63 1023,254,63   29786112   30298110     511999  249M  6 FAT16
[root@100ask_am335x:~]#
```

从上图可知/dev/sda2 是 FAT16 格式，挂载时可以指定类型为“vfat”：

```
[root@100ask_am335x:/sys/class/leds]# mount -t vfat /dev/sda2 /mnt
[ 8911.124996] FAT-fs (sda2): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
[root@100ask_am335x:/sys/class/leds]# ls /mnt/
EFI                     System Volume Information  WEPE
[root@100ask_am335x:/sys/class/leds]#
```

测试完以后，通过 umount 卸载/mnt，才可拔下 usb 设备：

```
[root@100ask_am335x:~]# umount   /mnt
```

5.3 OTG 接口测试

此节演示如何测试 OTG 接口的两种模式，分别是 device 模式和 host 模式。

注意：需要准备一个 OTG 转接头(开发板清单中不配)、micro usb 数据线(开发板清单里配有)。这 2 种设备如下图所示：



图 5.3.1 OTG 转换头



图 5.3.2 micro usb 线

1) otg device 模式测试：

开发板作为 USB 从设备，接到电脑上；电脑会把开发板识别为一个 U 盘。

先执行以下命令：

```
[root@100ask_am335x:~]# modprobe g_mass_storage file=/dev/mmcblk0p1 removable=1
[ 9249.536528] Mass Storage Function, version: 2009/09/11
[ 9249.541920] LUN: removable file: (no medium)
[ 9249.550287] LUN: removable file: /dev/mmcblk0p1
[ 9249.555672] Number of LUNs=1
[ 9249.558931] g_mass_storage gadget: Mass Storage Gadget, version: 2009/09/11
[ 9249.566829] g_mass_storage gadget: userspace failed to provide iSerialNumber
[ 9249.574393] g_mass_storage gadget: g_mass_storage ready
[root@100ask_am335x:~]# [ 9274.726921] g_mass_storage gadget: high-speed config #1: Linux File-Backed Storage

[root@100ask_am335x:~]#
```

然后使用 micro usb 线小的一头插入如下图 5.3.1 所示的蓝色框所在接口，另一端连接电脑 USB 接口，micro otg 接口位置如下图 5.3.1 所示：

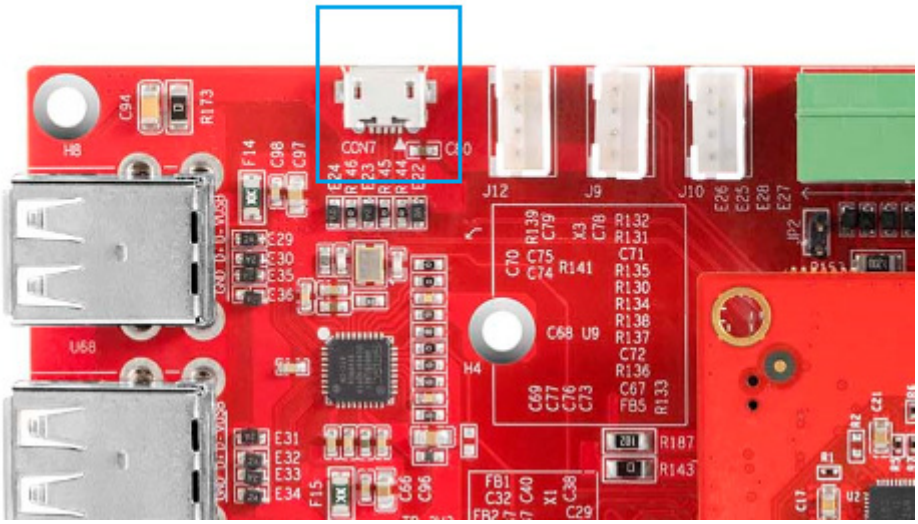
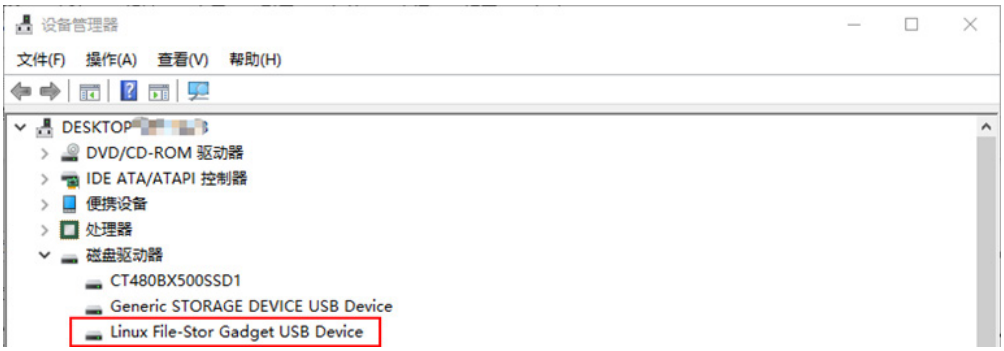


图 5.3.1 otg device 模式连接示意图

此时在电脑的设备管理器中可以看到一个名为 linux File-Stor Gadget USB Device 的磁盘设备，在 windows 资源管理器中也可以看到有新的可移动设备盘符弹出：



测试完成后，在终端下执行“rmmod g_mass_storage ”卸载改模块驱动：

```
[root@100ask_am335x:~]# rmmod   g_mass_storage
```

2) otg host 模式：

开发板作为 usb 主设备，其他 USB 设备通过 otg 转接头插入开发板，开发板即可识别出这些 USB 外

设备。

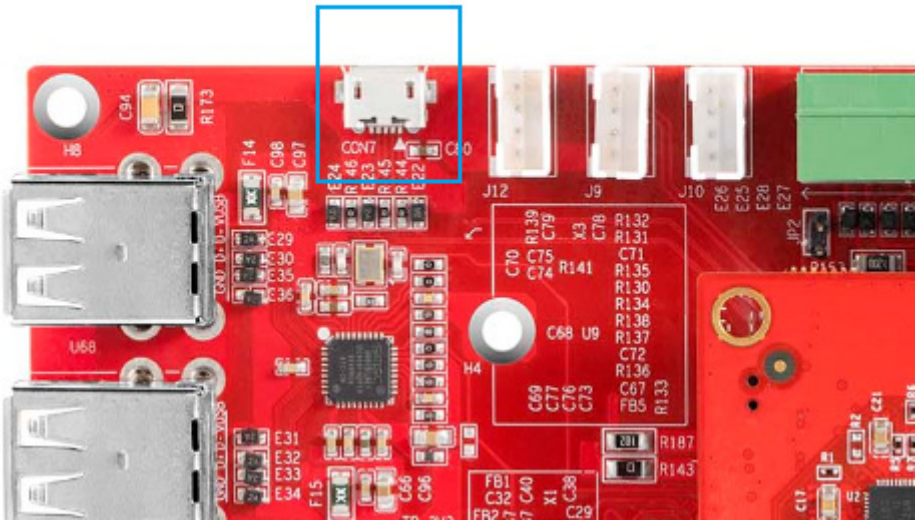


图 5.3.2.1 otg 转接头连接示意图

下图是把 U 盘通过 otg 转接头插入开发板后，在串口打印的信息：

```
[root@100ask_am335x:~]# [  985.063978] usb 1-1: new high-speed USB device number 2 using musb-hdrc
[  985.235046] usb 1-1: New USB device found, idVendor=13fe, idProduct=5500
[  985.242057] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[  985.249873] usb 1-1: Product: V60
[  985.253338] usb 1-1: Manufacturer: EAGET
[  985.257674] usb 1-1: SerialNumber: 070D5CA1380C8F92
[  985.266195] usb-storage 1-1:1.0: USB Mass Storage device detected
[  985.276725] scsi host0: usb-storage 1-1:1.0
[  986.325415] scsi 0:0:0:0: Direct-Access     EAGET    V60             PMAP PQ: 0 ANSI: 6
[  986.340585] sd 0:0:0:0: [sda] 30302208 512-byte logical blocks: (15.5 GB/14.4 GiB)
[  986.355177] sd 0:0:0:0: [sda] Write Protect is off
[  986.360600] sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[  986.515164]  sda: sda1 sda2
[  986.532491] sd 0:0:0:0: [sda] Attached SCSI removable disk

[root@100ask_am335x:~]#
```

通过打印的设备信息可知，系统为该 usb 存储设备创建的设备节点为 /dev/sda。一般来说/dev/sda 对应整个 U 盘，/dev/sda1 对应该 U 盘的第 1 个分区，/dev/sda2 对应第 2 个分区。

有些 U 盘没有划分分区，它只有一个设备节点/dev/sda，而没有/dev/sda1 等节点。对于这种情况，/dev/sda 既代表整个 U 盘，也代表第 1 个分区。

我们可以挂载某个分区，挂载之前要先通过 fdisk 命令获取分区类型,如下所示：

```
[root@100ask_am335x:~]# fdisk -l /dev/sda
Disk /dev/sda: 14 GB, 15514730496 bytes, 30302208 sectors
1886 cylinders, 255 heads, 63 sectors/track
Units: cylinders of 16065 * 512 = 8225280 bytes

Device  Boot StartCHS    EndCHS        StartLBA     EndLBA    Sectors  Size Id Type
/dev/sda1 *  33,37,5     1023,254,63     532480   29786111   29253632 13.9G  7 HPFS/NTFS
/dev/sda2    1023,254,63 1023,254,63   29786112   30298110     511999  249M  6 FAT16
[root@100ask_am335x:~]#
```

从上图可知/dev/sda2 是 FAT16 格式，挂载是可以指定类型为“vfat”：

```
[root@100ask_am335x:/sys/class/leds]# mount -t vfat /dev/sda2 /mnt
[ 8911.124996] FAT-fs (sda2): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.
[root@100ask_am335x:/sys/class/leds]# ls /mnt/
EFI                     System Volume Information  WEPE
[root@100ask_am335x:/sys/class/leds]#
```

测试完以后，通过 umount 卸载/mnt，才可拔下 usb 设备：

```
[root@100ask_am335x:~]# umount   /mnt
```

5.4 耳机接口测试

此节演示使用三段式耳机在 100ask_am335x 开发板上录制声音、播放音频。

注意：需要准备一个带麦克风的三段式耳机，如下图 5.4.1 所示：



图 5.4.1 三段式耳机示意图

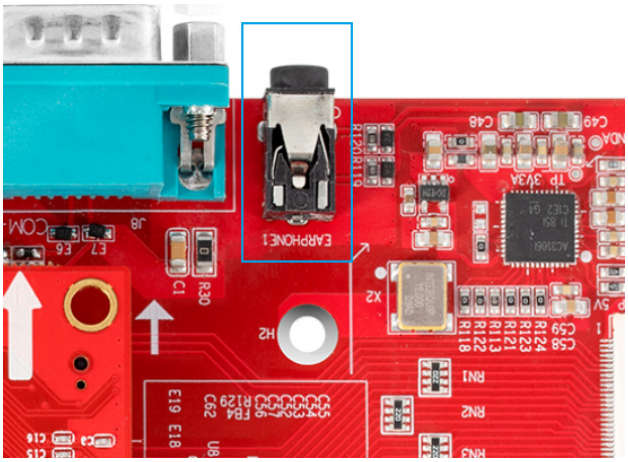


图 5.4.2 耳机连接位置示意图

1) 录制音频：

将耳机插入开发板耳机孔，使用如下命令进行录制(执行命令后，对着麦克风说话)：

```
[root@100ask_am335x:~]# arecord -v   --format=cd --device=plughw:0,0   test.wav
```

```
[root@100ask_am335x:~]# arecord  --format=cd --device=plughw:0,0 -v  test.wav
Recording WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
Plug PCM: Hardware PCM card 0 'AM335x-EVM' device 0 subdevice 0
Its setup is:
  stream       : CAPTURE
  access       : RW_INTERLEAVED
  format       : S16_LE
  subformat    : STD
  channels     : 2
  rate         : 44100
  exact rate   : 44100 (44100/1)
  msbits       : 16
  buffer_size  : 22052
  period_size  : 5513
  period_time  : 125011
  tstamp_mode  : NONE
  tstamp_type  : MONOTONIC
  period_step  : 1
  avail_min    : 5513
  period_event : 0
  start_threshold  : 1
  stop_threshold   : 22052
  silence_threshold: 0
  silence_size : 0
  boundary     : 1445199872
  appl_ptr     : 0
  hw_ptr       : 0
^CAborted by signal Interrupt...
[root@100ask_am335x:~]#
```

参数讲解：

- --format=cd ： 设置格式为 16 bit little endian, 44100, stereo
- test.wav ：指定录音文件的名称以及格式。其中 test 是文件名称，wav 是音频格式。支持的格式有 wav、raw 和 au 等。

2) 播放音频：

将耳机插入开发板耳机孔，使用 aplay 进行播放音频文件：

```
[root@100ask_am335x:~]# aplay -v--format=cd  --device=plughw:0,0  test.wav
```

```
[root@100ask_am335x:~]# aplay  --format=cd  --device=plughw:0,0 -v  test.wav
Playing WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
Plug PCM: Hardware PCM card 0 'AM335x-EVM' device 0 subdevice 0
Its setup is:
  stream       : PLAYBACK
  access       : RW_INTERLEAVED
  format       : S16_LE
  subformat    : STD
  channels     : 2
  rate         : 44100
  exact rate   : 44100 (44100/1)
  msbits       : 16
  buffer_size  : 22052
  period_size  : 5513
  period_time  : 125011
  tstamp_mode  : NONE
  tstamp_type  : MONOTONIC
  period_step  : 1
  avail_min    : 5513
  period_event : 0
  start_threshold  : 22052
  stop_threshold   : 22052
  silence_threshold: 0
  silence_size : 0
  boundary     : 1445199872
  appl_ptr     : 0
  hw_ptr       : 0
[root@100ask_am335x:~]#
```

参数讲解：

- --format=cd ： 设置格式为 16 bit little endian, 44100, stereo
- test.wav ：指定录音文件的名称以及格式。其中 test 是文件名称，wav 是音频格式。支持的格式有 wav、raw 和 au 等。

还可以通过 ssh 登录开发板，将电脑中的 wav 格式的音频上传到开发板，再用用 aplay 进行播放。

5.5 LCD 显示测试

此节演示通过 fb-test 测试程序让 lcd 显示红绿蓝白 4 中颜色，用以观察 lcd 的显示效果。

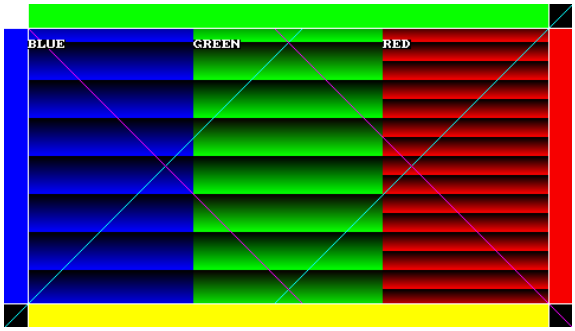
1) lcd 显示红色：

```
[root@100ask_am335x:~]# fb-test  -r
fb-test 1.1.0 (rosetta)
fb res 480x272 virtual 480x272, line_len 960, bpp 16
```



2) lcd 显示多种颜色：

```
[root@100ask_am335x:~]# fb-test f
fb-test 1.1.0 (rosetta)
fb res 480x272 virtual 480x272, line_len 960, bpp 16
```



5.6 触摸屏校准测试

此节演示通过 ts_calibrate 程序校准触摸屏，用以验证触摸屏是否可用。

在终端里运行 ts_calibrate 程序，此时就能看到 lcd 上依次弹出“+” 图标，依次点击此图标中心：

```
[root@100ask_am335x:~]# ts_calibrate
```

```
[root@100ask_am335x:~]# ts_calibrate
xres = 480, yres = 272
Took 2 samples...
Top left : X =   75 Y =   37
Took 1 samples...
Top right : X =  469 Y =   53
[   54.371144] random: crng init done
Took 1 samples...
Bot right : X =  471 Y =  228
Took 2 samples...
Bot left : X =   72 Y =  225
Took 6 samples...
Center : X =  269 Y =  144
-20.370605 0.958226 0.003637
12.270599 -0.022429 0.944776
Calibration constants: -1335008 62798 238 804166 -1469 61916 65536
[root@100ask_am335x:~]#
```



如果校准通过，最后会生成较准文件/etc/pointercal。

这时，可以执行 ts_test 命令使用触摸屏来在 LCD 上画线。

5.7 屏幕背光调节

此节演示通过操作 LCD 在/sys 目录下的对应文件，以实现查询、调节背光亮度。

背光亮度的设置范围为 0～8，0 表示最暗，8 表示最亮。

先通过 cat 命令查看当前背光亮度等级：

```
[root@100ask_am335x:~]# cat    /sys/class/backlight/backlight/brightness
```

```
[root@100ask_am335x:~]# cat /sys/class/backlight/backlight/brightness
4
[root@100ask_am335x:~]#
```

然后查看背光最大亮度等级：

```
[root@100ask_am335x:~]# cat /sys/class/backlight/backlight/max_brightness
```

```
[root@100ask_am335x:~]# cat /sys/class/backlight/backlight/max_brightness
8
[root@100ask_am335x:~]#
```

最后设置背光亮度值为 8，同时可以观察屏幕背光亮度是否有变化：

```
[root@100ask_am335x:~]# echo 8 > /sys/class/backlight/backlight/brightness
```

```
[root@100ask_am335x:~]# echo 8 > /sys/class/backlight/backlight/brightness
[root@100ask_am335x:~]#
```

5.8 RTC 测试

此节演示通过使用 date 和 hwclock 命令设置系统时间、硬件时间，并测试当操作系统重启后，系统时钟与硬件时间是否同步。

一般的板子都会有一个名为 RTC(实时时钟)的硬件，RTC 使用钮扣电池供电，在系统关闭时用来维持时钟。RTC 维持的时间，被称为硬件时间。

图 5.8.1 中，显示的是 RTC 纽扣电池的安装位置。

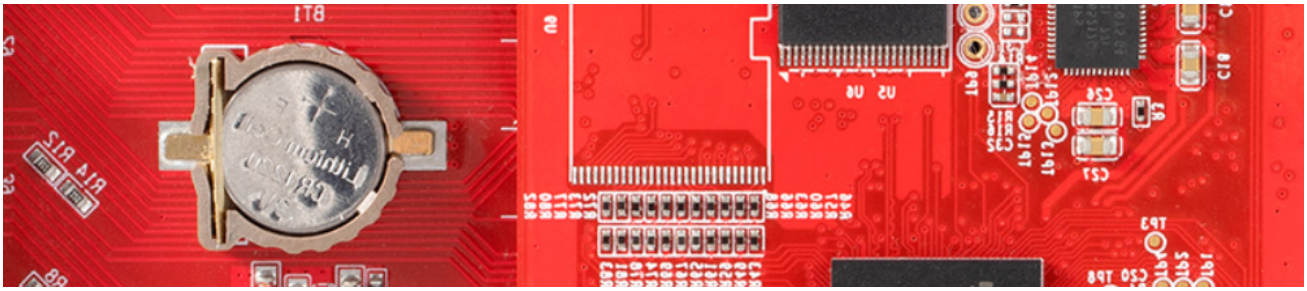


图 5.8.1 RTC 位置示意图

Linux 系统启动之后，它会自己维持时间，这个时间被称为系统时间。系统时间的初始值来源有二：

① 如果没有 RTC，系统时间初始值为 1970 年 1 月 1 日 0 点 0 分 0 秒

② 如果有 RTC，Linux 启动后，系统时间初始值从 RTC 读取

在实际使用过程中，要注意系统时间、硬件时间的同步问题：

① 使用 date 命令查看、设置系统时间，在设置系统时间后，要使用“hwclock -w” 命令同步到 RTC；

② 使用 hwclock 查看、设置硬件时间，在设置硬件时间后，要使用“hwclock -s”命令同步到系统时间。

注意：如果要使用 RTC，要确保板子上已经安装了纽扣电池。

以下命令是设置系统时间、并同步到 RTC：

```
[root@100ask_am335x:~]# date -s "2019-06-18 12:00:00"
[root@100ask_am335x:~]# hwclock -w
```

```
[root@100ask_am335x:~]# date -s "2019-06-18 12:00:00"
Tue Jun 18 12:00:00 UTC 2019
[root@100ask_am335x:~]# hwclock -w
[root@100ask_am335x:~]#
```

一般不直接设置硬件时间，要设置硬件时间时，先使用 date 设置系统时间，再使用“hwclock -w”同步到 RTC 硬件。

你使用 date、hwclock 命令设置好时间后，可以关闭开发板并等待一会后重启，再用 date 命令查看时间是否正常。

对 RTC 硬件的操作使用 hwclock 命令，常见用法如下：

- hwclock -r ：显示硬件时钟与日期
- hwclock -s ：将系统时钟调整为与目前的硬件时钟一致。
- hwclock -w ：将硬件时钟调整为与目前的系统时钟一致。

下面是一个示例，读取硬件时间：

```
[root@100ask_am335x:~]# hwclock -r
Tue Jun 18 12:06:58 2019  0.000000 seconds
[root@100ask_am335x:~]# hwclock -r
Tue Jun 18 12:06:59 2019  0.000000 seconds
[root@100ask_am335x:~]# hwclock -r
Tue Jun 18 12:07:00 2019  0.000000 seconds
[root@100ask_am335x:~]#
```

5.9 按键测试

此节演示通过 hexdump 命令以及 dmesg 命令来查看按键是否有反应，按键位置，以及对应的标识如下图 5.9.1 所示



图 5.9.1 按键位置示意图

执行下面命令之后，操作按键，如果一切正常会有打印信息：

```
[root@100ask_am335x:~]# hexdump   /dev/input/event1
```

```
[root@100ask_am335x:~]# hexdump  /dev/input/event1
0000000 017e 0000 7a52 0007 0001 0001 0001 0000
0000010 017e 0000 7a52 0007 0000 0000 0000 0000
0000020 017e 0000 fd4a 000a 0001 0001 0000 0000
0000030 017e 0000 fd4a 000a 0000 0000 0000 0000
0000040 0181 0000 af34 000a 0001 001c 0001 0000
0000050 0181 0000 af34 000a 0000 0000 0000 0000
0000060 0181 0000 0e2d 000e 0001 001c 0000 0000
0000070 0181 0000 0e2d 000e 0000 0000 0000 0000
0000080 0183 0000 6d31 0004 0001 006a 0001 0000
0000090 0183 0000 6d31 0004 0000 0000 0000 0000
00000a0 0183 0000 7a4e 0007 0001 006a 0000 0000
00000b0 0183 0000 7a4e 0007 0000 0000 0000 0000
00000c0 0184 0000 6d27 0004 0001 0069 0001 0000
00000d0 0184 0000 6d27 0004 0000 0000 0000 0000
00000e0 0184 0000 7a3f 0007 0001 0069 0000 0000
00000f0 0184 0000 7a3f 0007 0000 0000 0000 0000
^C
[root@100ask_am335x:~]#
```

当然也可以执行下面的命令，查看内核的打印信息：

```
[root@100ask_am335x:~]# dmesg | grep "Dev: input1"
```

```
[root@100ask_am335x:~]# dmesg | grep "Dev: input1"
[  176.594047] evbug: Event. Dev: input1, Type: 1, Code: 1, Value: 1
[  176.594057] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  176.824006] evbug: Event. Dev: input1, Type: 1, Code: 1, Value: 0
[  176.824015] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  178.054045] evbug: Event. Dev: input1, Type: 1, Code: 28, Value: 1
[  178.054060] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  178.313986] evbug: Event. Dev: input1, Type: 1, Code: 28, Value: 2
[  178.313994] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 1
[  178.334009] evbug: Event. Dev: input1, Type: 1, Code: 28, Value: 0
[  178.334017] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  179.494081] evbug: Event. Dev: input1, Type: 1, Code: 106, Value: 1
[  179.494098] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  179.704017] evbug: Event. Dev: input1, Type: 1, Code: 106, Value: 0
[  179.704024] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  180.784084] evbug: Event. Dev: input1, Type: 1, Code: 105, Value: 1
[  180.784099] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[  181.043990] evbug: Event. Dev: input1, Type: 1, Code: 105, Value: 2
[  181.043998] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 1
[  181.074020] evbug: Event. Dev: input1, Type: 1, Code: 105, Value: 0
[  181.074029] evbug: Event. Dev: input1, Type: 0, Code: 0, Value: 0
[root@100ask_am335x:~]#
```

5.10 LED 灯测试

此节演示通过操作 LED 在/sys 目录下的对应文件，以点亮、熄灭 LED，按键位置以及对应的按键标识如下图 5.10.1 所示



图 5.10.1led 灯位置示意图

注意: led1/led2/led3 灯已经被定义为系统状态灯，仍可在/sys 目录中操作对应文件；led4 留给用户用于其他目的。

首先，进入/sys/class/leds 目录查看在多少个 LED：

```
[root@100ask_am335x:~]# cd /sys/class/leds/
```

```
[root@100ask_am335x:~]# ls /sys/class/leds/
am335x:green:cpu0  am335x:green:mmc0  am335x:green:nand
[root@100ask_am335x:~]#
```

这些文件对应的 LED 是：

① am335x:green:mmc0：对应开发板 led1，用于显示 SD 卡的读写

② am335x:green:cpu0：对应开发板 led2，用于显示 CPU 的运行

③ am335x:green:nand：对应开发板 led3，用于显示 Nand Flash 的读写

```
[root@100ask_am335x:/sys/class/leds/am335x:green:mmc0]# ls
brightness      device          max_brightness  power           subsystem       trigger         uevent
[root@100ask_am335x:/sys/class/leds/am335x:green:mmc0]#
```

此时我们通过 echo 输入值到 brightness 即可控制灯的亮灭，以下以 am335x:green:mmc0 为示例

```
[root@100ask_am335x:/sys/class/leds/am335x:green:mmc0]# echo 1 > brightness   //灯亮
```

```
[root@100ask_am335x:/sys/class/leds/am335x:green:mmc0]# echo 0 > brightness   //灯灭
```

5.11 串口测试

此节演示使用 linux-serial-test 程序测试串口的可用性，如下所示，使用杜邦线短接黑色框框内的引脚即可进行回环测试。

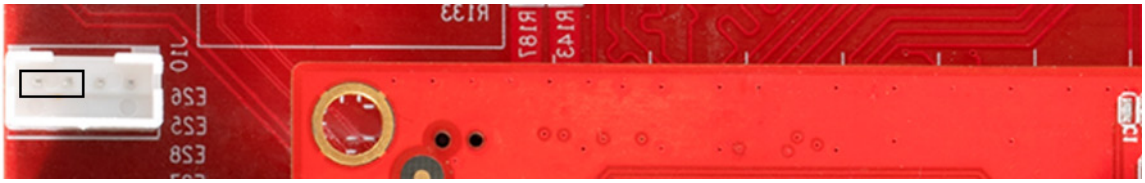


图 5.11.1 串口测试连接示意图

1) 压力测试：

这发送五秒钟并接收七秒钟，之后它将退出。如果接收的字节数与发送的字节数匹配，并且接收的模式是正确的，则退出代码将为零(ret=0)，因此可以将其用作自动测试脚本的一部分。

```
[root@100ask_am335x:~]# linux-serial-est -s -e -p /dev/ttyO2 -b 115200 -o 5 -i 7
[root@100ask_am335x:~]# linux-serial-test -s -e -p /dev/ttyO2 -b 115200 -o 5 -i 7
Linux serial test app
Stopped transmitting.
/dev/ttyO2: count for this session: rx=57903, tx=57903, rx err=0
/dev/ttyO2: TIOCGICOUNT: ret=0, rx=730770, tx=767105, frame = 0, overrun = 0, parity = 0, brk = 0, buf_overrun = 0
Stopped receiving.
/dev/ttyO2: count for this session: rx=57903, tx=57903, rx err=0
/dev/ttyO2: TIOCGICOUNT: ret=0, rx=730770, tx=767105, frame = 0, overrun = 0, parity = 0, brk = 0, buf_overrun = 0
[root@100ask_am335x:~]#
```

2) 全速压力测试：

```
[root@100ask_am335x:~]# linux-serial-test -s -e -p /dev/ttyO2 -b 3000000
[root@100ask_am335x:~]# linux-serial-test -s -e -p /dev/ttyO2 -b 3000000
Linux serial test app
```

此时使用 top 命令查看系统运行状态，可以看出 cpu 的使用率，一直处于高负载状态。

参数详解

- -b， 串口波特率，115200 等（默认为 115200）
- -p， port Port（/ dev / ttyS0 等）（必须指定）
- -s， stats 每 5 秒转储串口数据
- -e， dump-err 显示错误
- -o， tx-time 要传输的秒数（默认为 0，表示没有限制）
- -i， rx-time 要接收的秒数（默认为 0，表示没有限制）

5.12 RS485 测试

此节演示使用 rs485_test 程序测试 rs485 接口。

注意：rs485 通信时需要 2 个 rs485 设备，使用 3.5 15EDG 5P 插拔式接线端子进行连接测试，以下测试是通过两块 335x 开发板进行测试，默认配置不含接线端子，请自行购买连接测试。

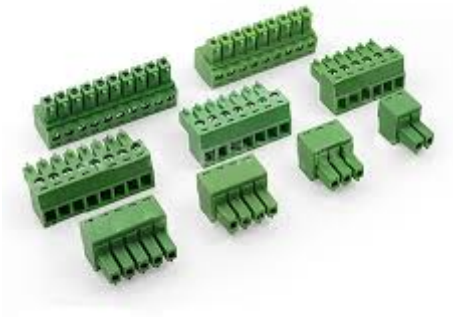


图 5.12.1 RS485 连接端子示意图

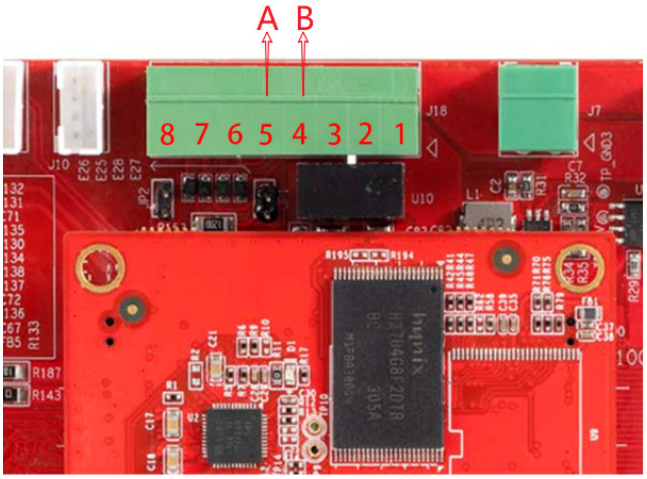


图 5.12.2 RS485 测试开发板连接位置示意图

1) 接收端：

```
[root@100ask_am335x:~]# rs485_test -d /dev/ttyO2 -b 115200
```

```
[root@100ask_am335x:~]# rs485_test  -d  /dev/ttyO2  -b 115200
/dev/ttyO2 SEND: 1234567890
/dev/ttyO2 RECV 1 total
/dev/ttyO2 RECV: 1
/dev/ttyO2 RECV 8 total
/dev/ttyO2 RECV: 23456789
/dev/ttyO2 RECV 1 total
/dev/ttyO2 RECV: 0
/dev/ttyO2 SEND: 1234567890
/dev/ttyO2 RECV 1 total
/dev/ttyO2 RECV: 1
/dev/ttyO2 RECV 2 total
/dev/ttyO2 RECV: 23
/dev/ttyO2 RECV 1 total
/dev/ttyO2 RECV: 4
/dev/ttyO2 RECV 1 total
/dev/ttyO2 RECV: 5
```

2) 发送端：

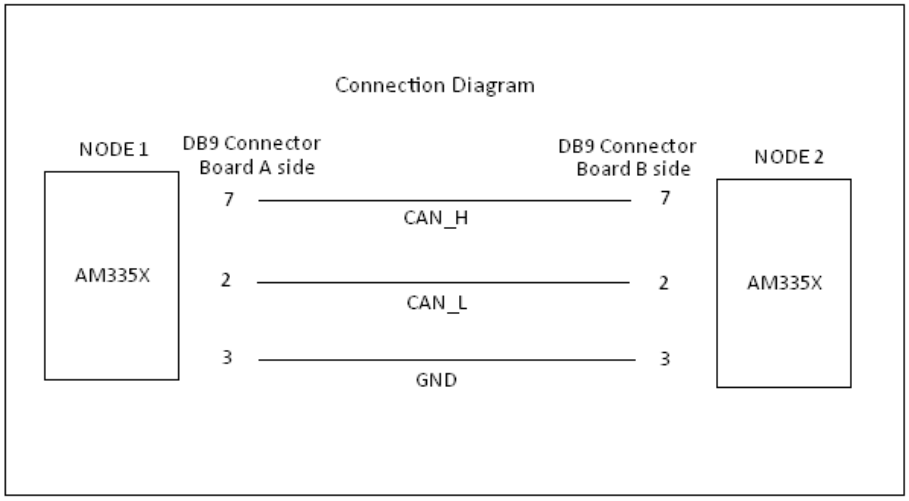
```
[root@100ask_am335x:~]# rs485_test -d /dev/ttyO1 -b 115200
```

```
[root@100ask_am335x:~]# rs485_test -d  /dev/ttyO1  -b 115200
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 SEND: 1234567890
/dev/ttyO1 RECV 1 total
/dev/ttyO1 RECV: 1
/dev/ttyO1 RECV 9 total
/dev/ttyO1 RECV: 234567890
/dev/ttyO1 SEND: 1234567890
```

5.13 CAN 功能测试

此节主要演示使用两块开发板通过 ip 和 can-utils 命令测试 can0 的通信。

CAN 通信需要 2 个设备，本节使用两块 100ask_am335x 开发板做测试。你需要根据原理图找到 CAN 接口引脚，用 3.5 15EDG 5P 插拔式接线端子连接线连接两个 CAN 接口的 H、L、GND 三条线。用户也可以使用自己的 Can 设备进行测试。



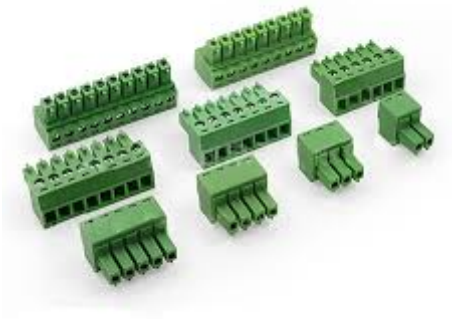


图 5.13.1 CAN 接线端子

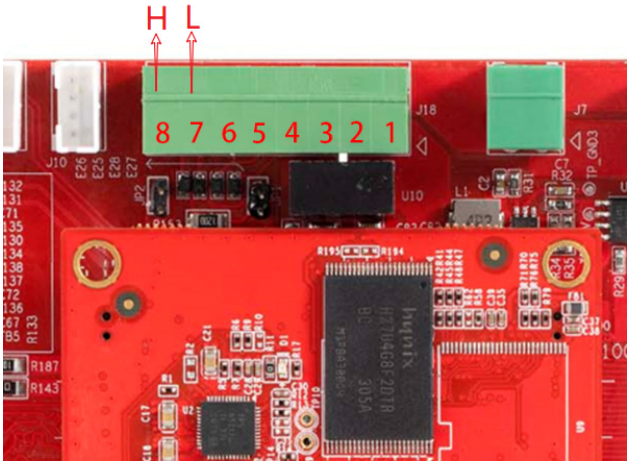


图 5.13.2　CAN 测试开发板连接示意图

1) 发送端：

关闭 can0 接口：

```
[root@100ask_am335x:~]# ip link set can0 down
```

设置 can0 传输速率为 50Kbits/sec：

```
[root@100ask_am335x:~]# ip link set can0 type can bitrate 50000
```

打印 can0 的信息：

```
[root@100ask_am335x:~]# ip -details link show can0
```

打开 can0 接口：

```
[root@100ask_am335x:~]# ip link set can0 up
```

使用 cansend 命令向另一端发送数据：

```
[root@100ask_am335x:~]# cansend can0 123#DEADBEEF
[root@100ask_am335x:~]# cansend can0 123#DEADBEEF
[root@100ask_am335x:~]# cansend can0 123#DEADBE23
```

下图是一个操作示例(建议等接收端设置好并执行 candump 命令之后，发送端再执行 cansend 命令)：

```
[root@100ask_am335x:~]# ip link set can0 down
[root@100ask_am335x:~]# ip link set can0 type can bitrate 50000
[root@100ask_am335x:~]# ip -details link show can0
2: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can  promiscuity 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
          bitrate 50000 sample-point 0.875
          tq 1250 prop-seg 6 phase-seg1 7 phase-seg2 2 sjw 1
          c_can: tseg1 2..16 tseg2 1..8 sjw 1..4 brp 1..1024 brp-inc 1
          clock 24000000numtxqueues 0 gso_max_size 0 gso_max_segs 0
[root@100ask_am335x:~]# ip link set can0 up
[  542.789779] c_can_platform 481d0000.can can0: setting BTR=1c1d BRPE=0000
[root@100ask_am335x:~]# cansend can0 123#DEADBEEF
[root@100ask_am335x:~]# cansend can0 123#DEADBEEF
[root@100ask_am335x:~]# cansend can0 123#DEADBE23
[root@100ask_am335x:~]#
```

2) 接收端

关闭 can0 接口：

```
[root@100ask_am335x:~]# ip link set can0 down
```

设置 can0 传输速率为 50Kbits/sec：

```
[root@100ask_am335x:~]# ip link set can0 type can bitrate 50000
```

打印 can0 的信息：

```
[root@100ask_am335x:~]# ip -details link show can0
```

打开 can0 接口：

```
[root@100ask_am335x:~]# ip link set can0 up
```

打印 can0 的接收到的数据：

```
[root@100ask_am335x:~]# candump can0
```

下图是一个操作示例，用来接收、显示数据：

```
[root@100ask_am335x:~]# ip -details link show can0
2: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can  promiscuity 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
          bitrate 50000 sample-point 0.875
          tq 1250 prop-seg 6 phase-seg1 7 phase-seg2 2 sjw 1
          c_can: tseg1 2..16 tseg2 1..8 sjw 1..4 brp 1..1024 brp-inc 1
          clock 24000000numtxqueues 0 gso_max_size 0 gso_max_segs 0
[root@100ask_am335x:~]# ip link set can0 up
[  542.789779] c_can_platform 481d0000.can can0: setting BTR=1c1d BRPE=0000
```

```
[root@100ask_am335x:~]# candump can0


  can0  123   [4]  DE AD BE EF
  can0  123   [4]  DE AD BE EF


  can0  123   [4]  DE AD BE 23

^C[root@100ask_am335x:~]# ^C
```

第6章 Qt 应用开发(仅供测试)

QT（发音为“cute”）是跨平台的应用程序框架和小部件工具包，用于创建经典和嵌入式图形用户界面，和应用上具有很少或没有变化的各种软件和硬件平台上运行在底层代码库中，仍然是具有本机功能和速度的本机应用程序。Qt 是目前正在开发的两个 Qt 的公司，一家上市公司，以及 Qt 的项目下的开源治理，包括个人开发者和工作推进 Qt 的公司。Qt 可在商业许可和开源 GPL 2.0，GPL 3.0 和 LGPL 3.0 许可下使用。

参考网址

- Qt 官网 https://www.qt.io/
- 维基百科 https://wiki.qt.io/
- Qt 下载 https://download.qt.io/archive/
- 参考网址 http://wiki.100ask.net/QT

注意：

① 安装 Qt 前序先使用 buildroot 编译 100ask_am335x_qt_defconfig 配置文件，生成包含 qt 环境的文件系统，具体编译过程请参考构建根文件系统，并将生成的系统烧写进 SD 卡，并启动。

② 百问网没有 QT 教程，无法对 QT 的技术问题提供支持；本章内容仅供参考。

③ 此安装教程只在百问网提供的 ubuntu18.04LTS 上进行过测试，其它版本请酌情修改。

6.1 安装 Qtcreator

QtCreator 下载网址：https://download.qt.io/official_releases/qtcreator/

这里我们使用的 qtcreator 版本为 4.8：

```
book@100ask:~$ cd /home/book
book@100ask:~$ cp /media/03-Tools/qt-creator-opensource-linux-x86_64-4.1.0.run   /home/book
book@100ask:~$ chmod a+x qt-creator-opensource-linux-x86_64-4.1.0.run
book@100ask:~$ sudo ./qt-creator-opensource-linux-x86_64-4.1.0.run
```

此安装过程类似 Windows 下应用的安装方法，一路点击下一步即可。

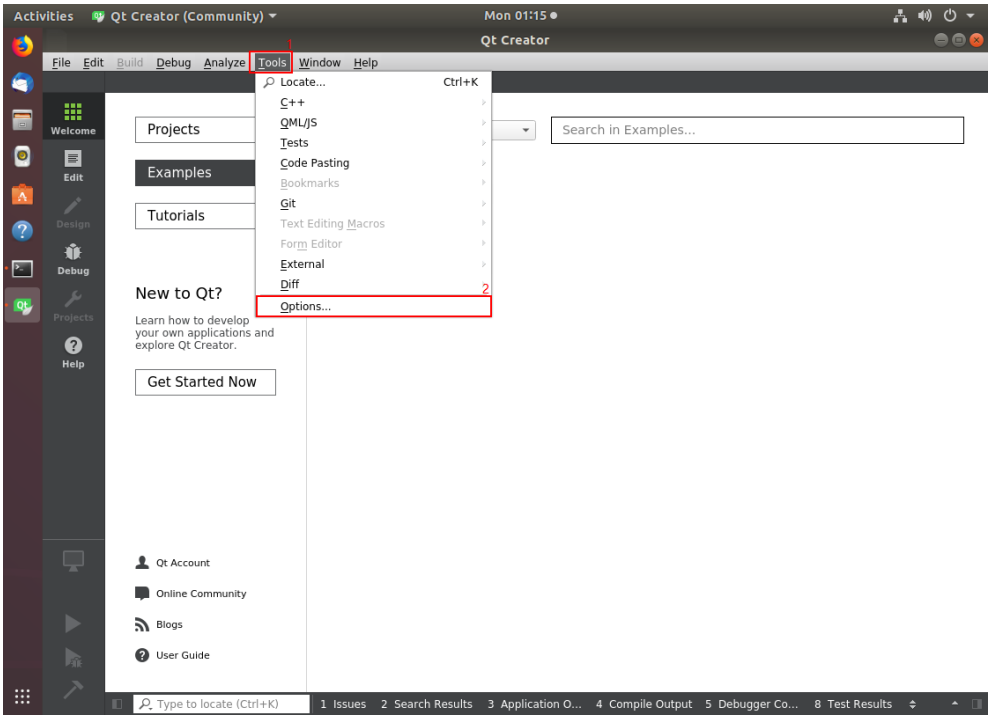
6.2 配置 QtCreator 开发环境

在 ubuntu 桌面打开终端，进入安装目录，运行如下命令启动 QtCreator：

```
book@100ask:~$ cd   /opt/qtcreator-4.8.0/bin
book@100ask: /opt/qtcreator-4.8.0/bin$ sudo ./qtcreator
```

1) 打开选项界面：

在 QtCreator 界面中，依次点击 **tools -> options**：



打开设置

2) 选择编译器：

在出现的选项对话框中，在左边点击**Kits**，右边选择**Compilers** 标签，并点击**Add**选择**Custom->C++**：

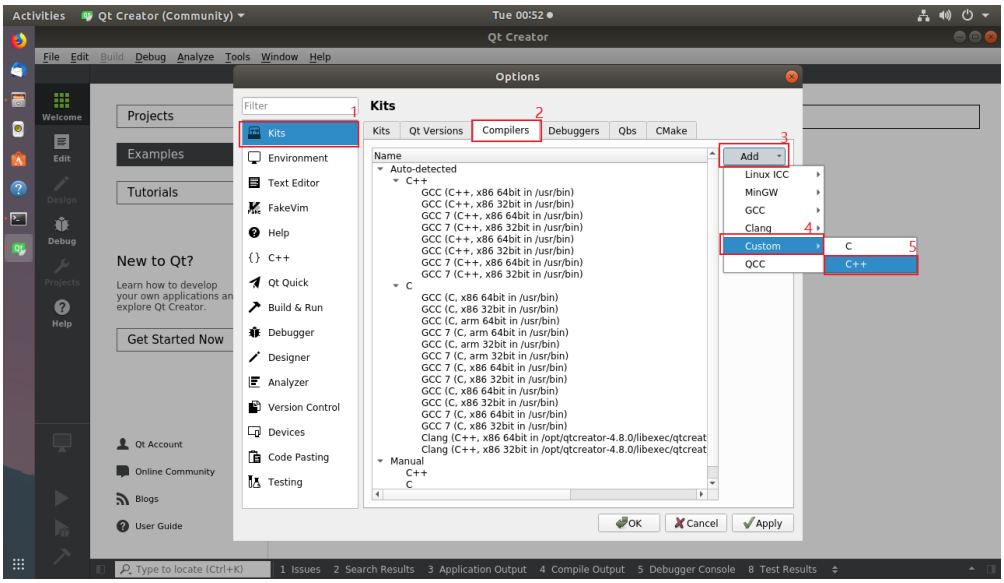


图5-2-1 编译器选择

3) 设置编译器：

在弹出的对话框中填写以下内容：**Compiler path**，**Make path** 和 **ABI**；填写完成后，点击 **Apply** 进行保存。

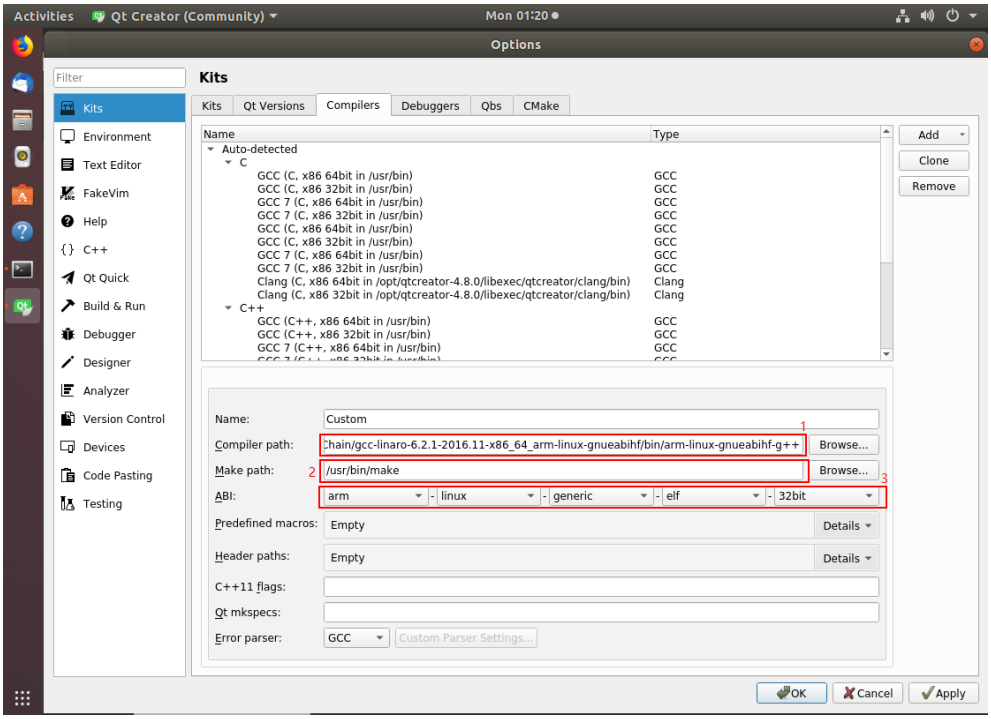


图 5-2-2 添加编译器

4) 添加 QT 版本：

在同一窗口下，选择 Qt Version 标签，在右侧点击“Add...”，会弹出对话框,切换目录到 Chapter3-4-1 中编译 QT 得到的 SDK 目录,选择 **qmake** 文件后,点击 **open** 按钮,设置完成之后，点击 **Apply** 按钮保存。

注意：qmake 文件是 buildroot 编译根文件系统后生成的，文件在 buildroot 目录下

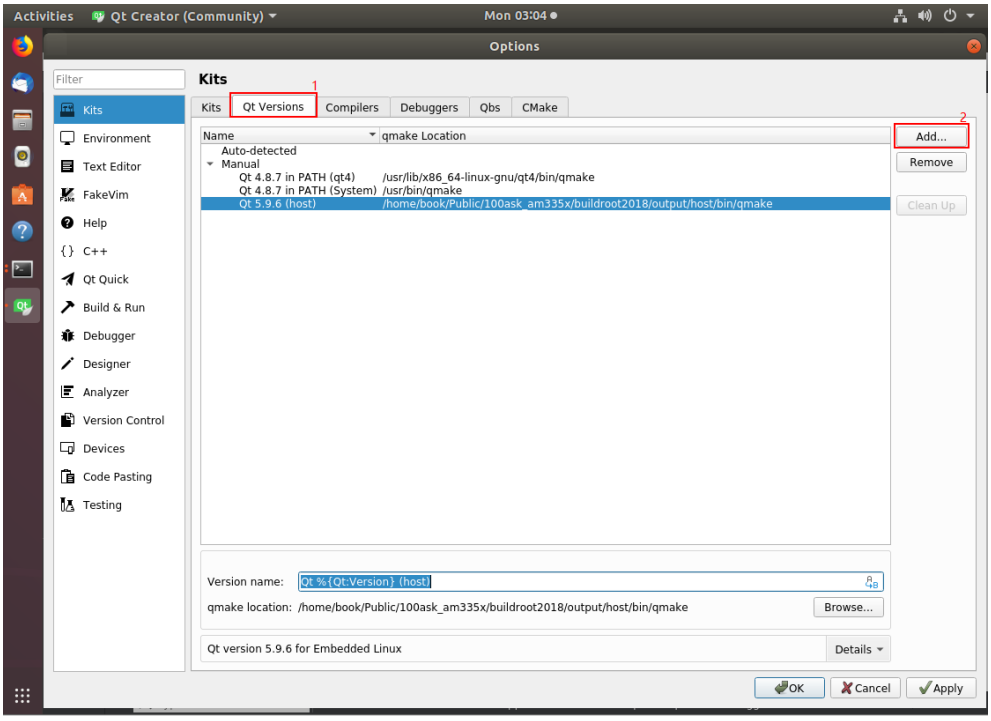


图 5-2-3 选择 qmake 构建工具

5) 综合选择：

继续选择上边的 Kits 标签，点击右侧 Add，填写相应内容：

① Name：输入 100ask_am335x

② Sysroot：输入交叉编译工具链的目录

③ compiler：c 和 c++这两个选择框里，都选择 Custom

④ Debugger：选择 None

⑤ Qtversio：选择上图中配置的“Qt5.9.6 (host) ”

配置完成后点击 apply,点击 OK。

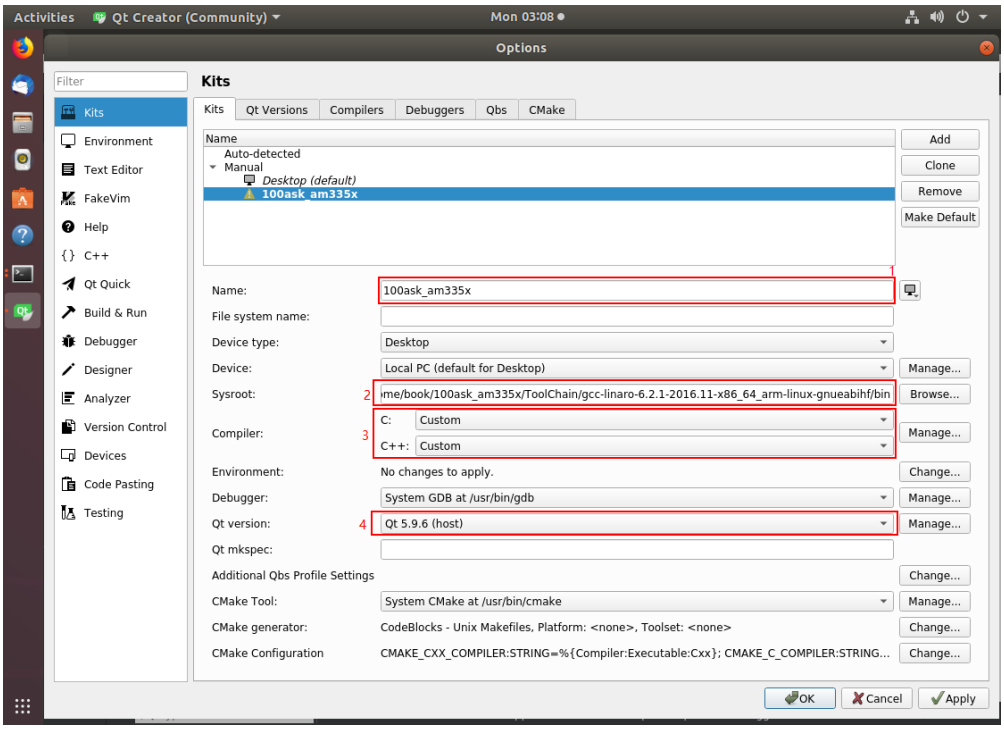


图 5-2-4 添加 Kits

6.3 创建 Helloworld 项目

1) 新建项目：

运行 QtCreator 后，在菜单栏选择 **File -> New File or Project**

2) 选择项目类型：

在打开的对话框中,依次选择 **Application -> Qt Widgets Application** ,点击 **Choose...** ，如下图所示:

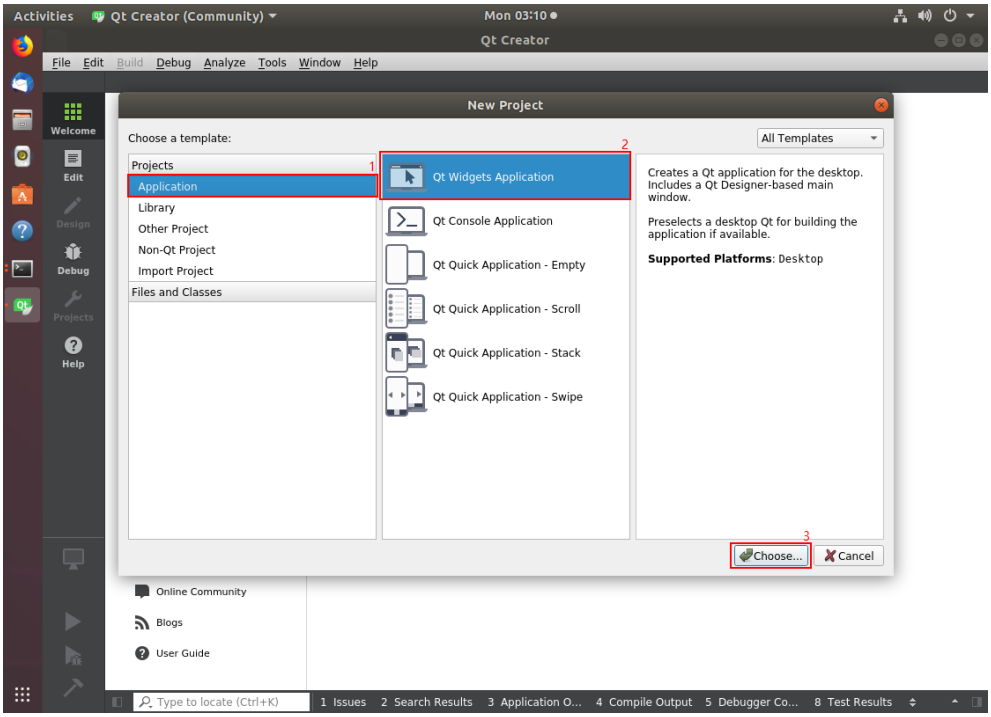


图 5-2-5 创建新工程

3) 输入项目名字、设置保存位置：

在弹出的 Qt Widgets Application 对话框中,设置项目名称为 helloword,Create in 一栏填写项目的存储路，点击 Next。如下：

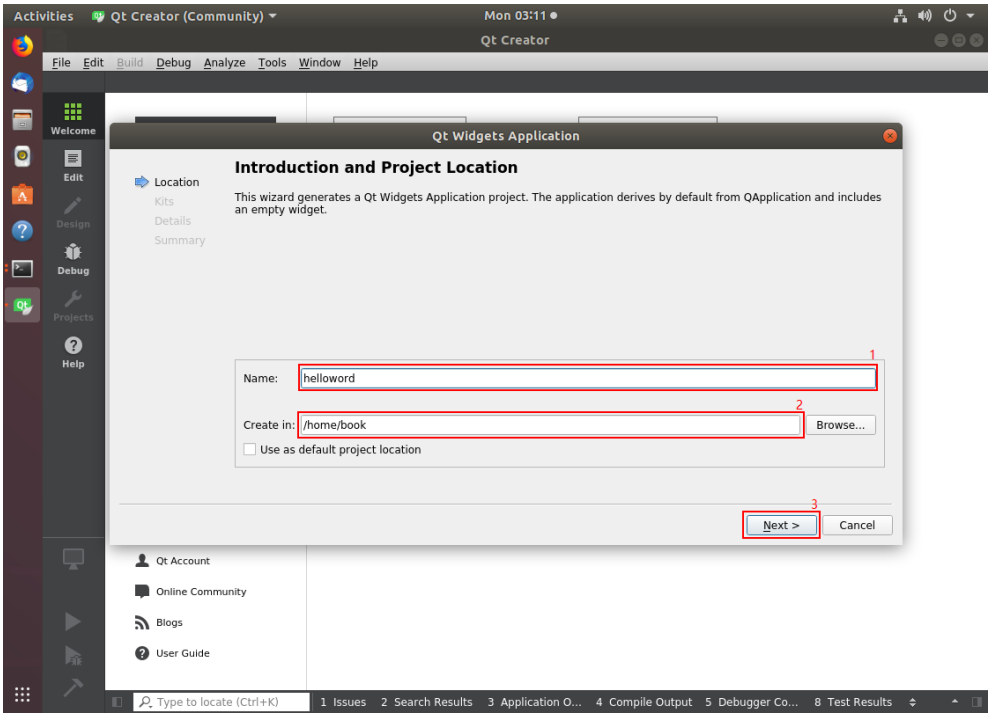


图 5-2-6 设置新工程名称和路径

4) 选择之前添加好的 Kits(100ask_am335x)，继续点击 Next，如下:

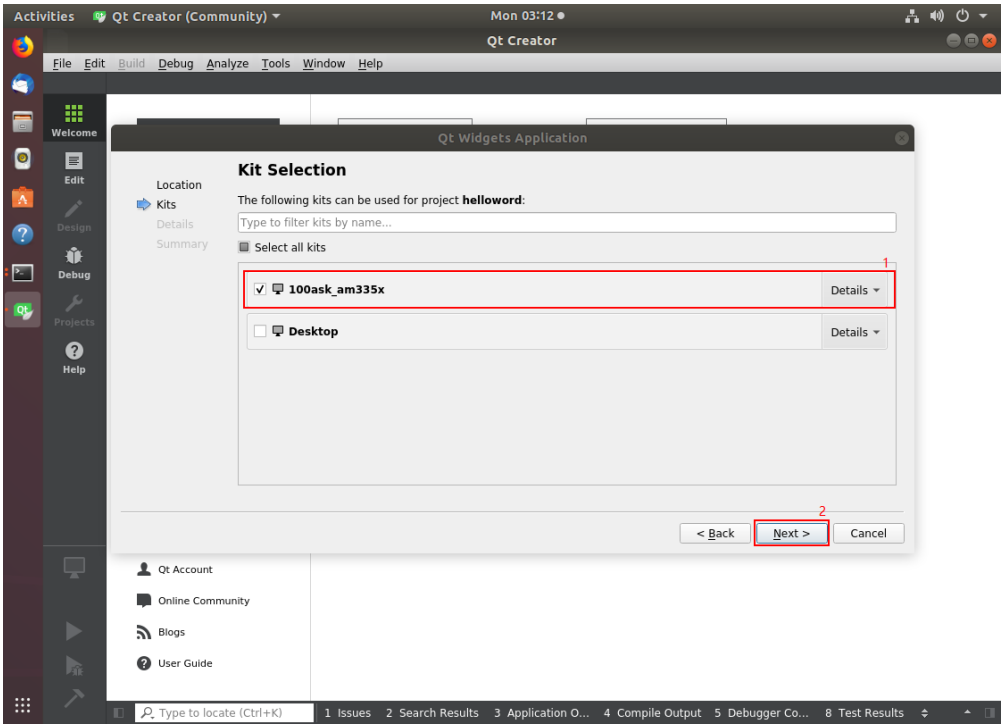


图 5-2-7 设置新工程 Kits

5) 选择基类：

当前的的应用继承自哪种 Widget,默认选择 QMainWindow,然后点击 **Next** 进入下一步。

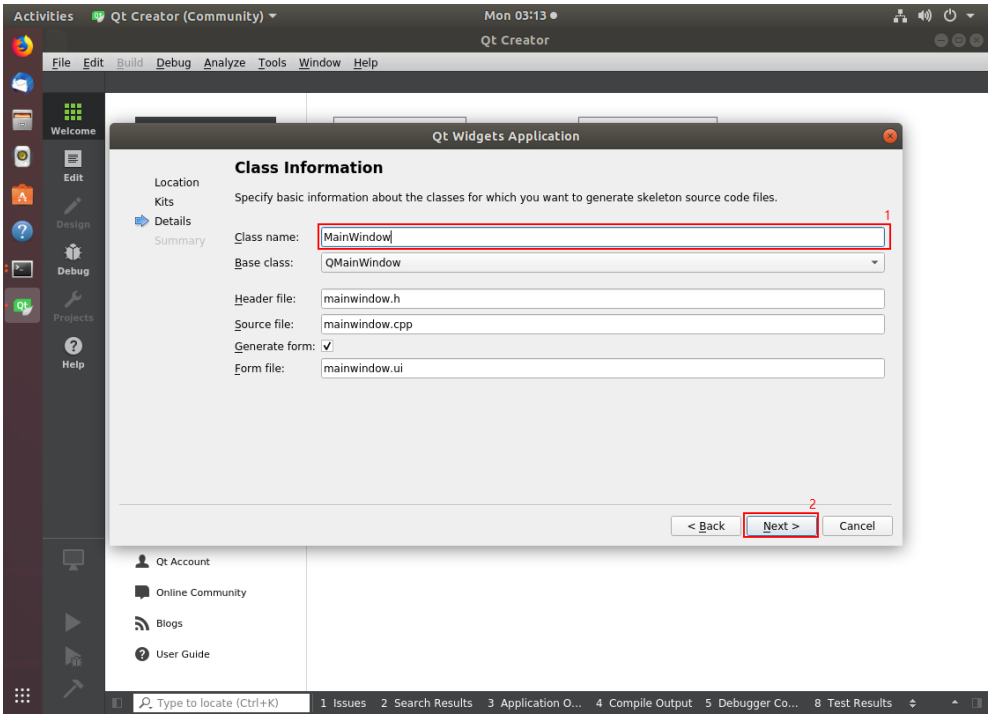


图 5-2-8 选择应用程序的类型

6) 完成项目创建：

以上信息填写完后,点击 Finish，完成 Helloworld 项目的创建。

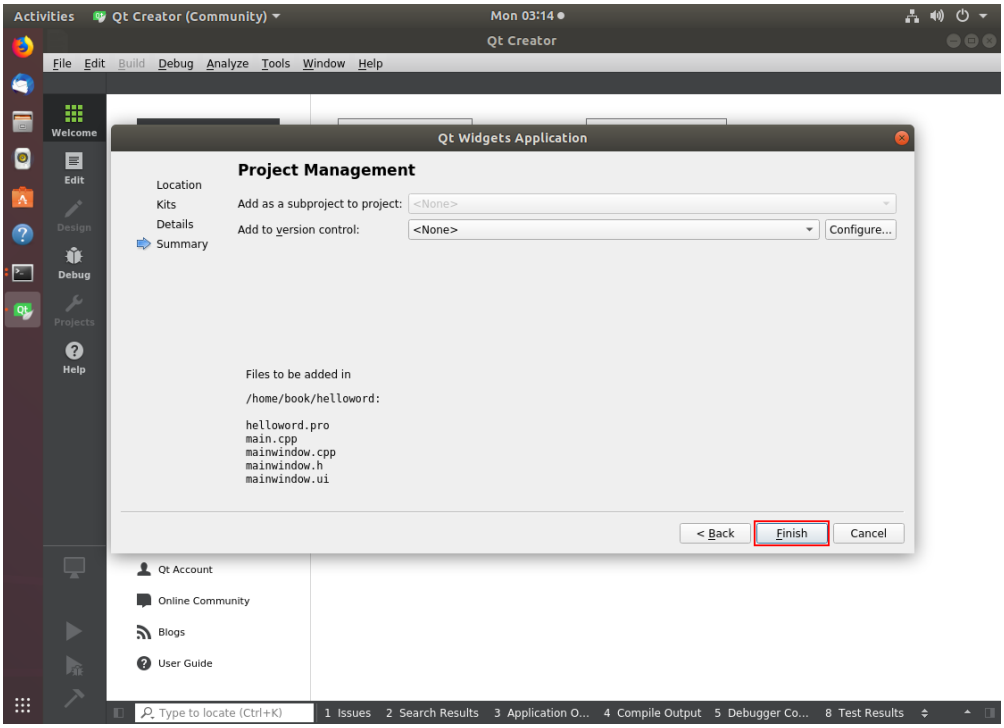


图 5-2-9 完成工程创建

6.4 编译 Qt 程序

以下是创建好的 helloworld 项目截图，左侧是 QtCreator 创建好的项目目录结构，右侧是代码编辑区，右下是编译时的输出信息：

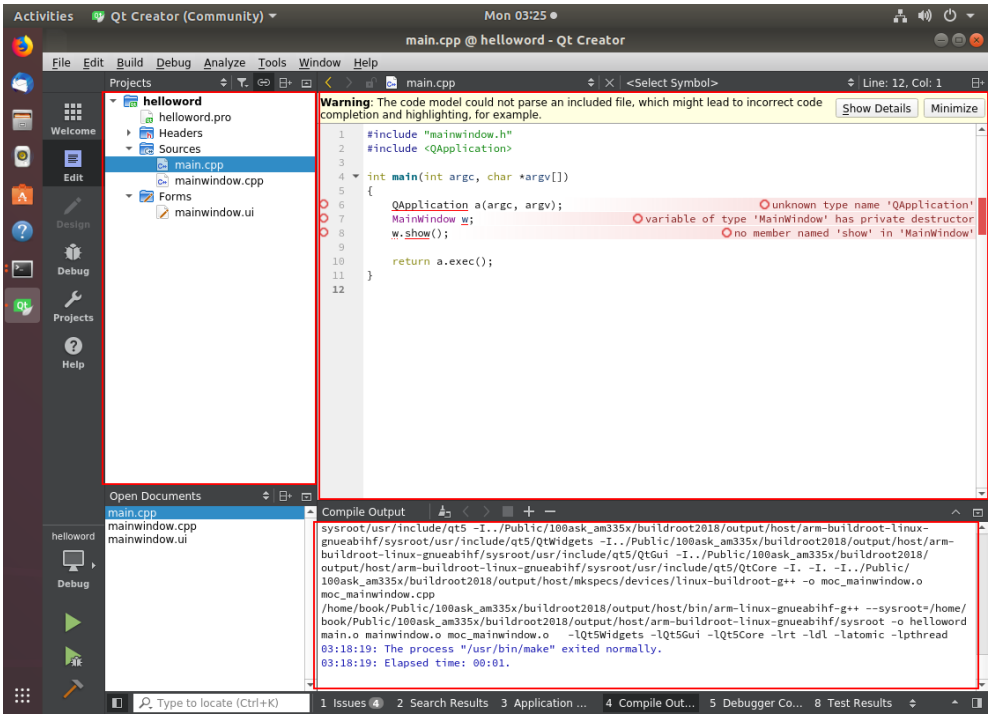


图 5-3-1 项目文件管理窗口

1) 修改界面：

在上面的图 5-3-1 界面中，双击左侧的 Forms 里的 mainwindow.ui 文件，打开 Design 视图。然后如下图所示，从左侧 Display Widgets 栏目下，拖动 Label 到中间的区域。

最后双击中间区域的 Label，修改内容为“Hello world! ”。

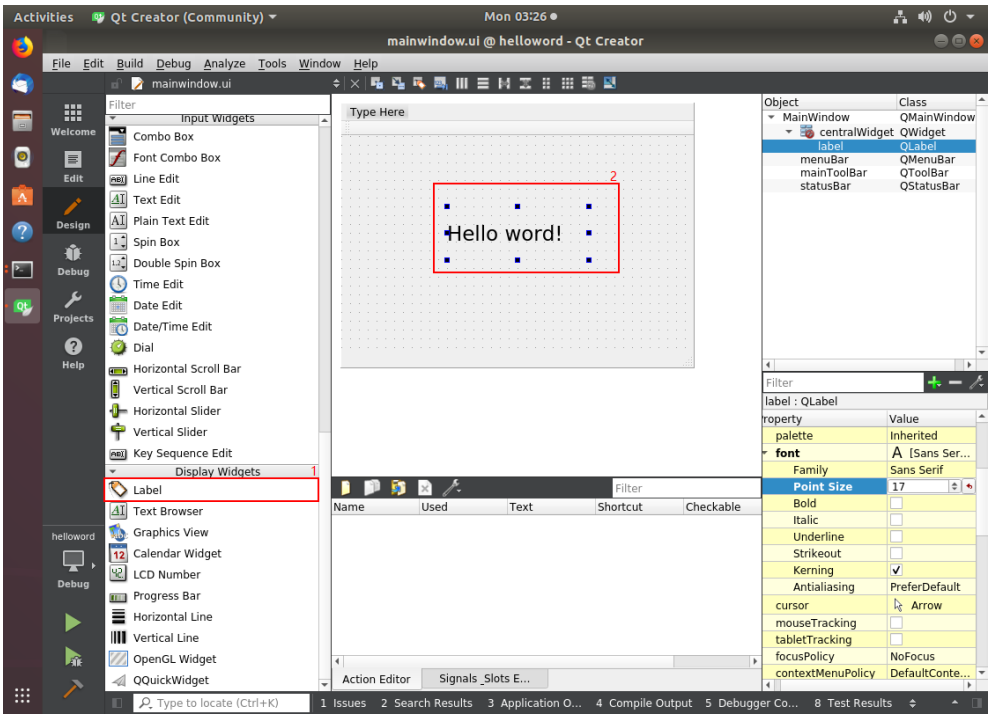


图 5-3-2 可视化界面编辑

2) 构建：

点击菜单栏 Build -> Build Project hellowrld，开始编译、构建项目。

在构建过程中，会在左下侧是“Compile Output”窗口打印构建信息。如果有错误，请根据提示出错信息修改，然后重新构建。

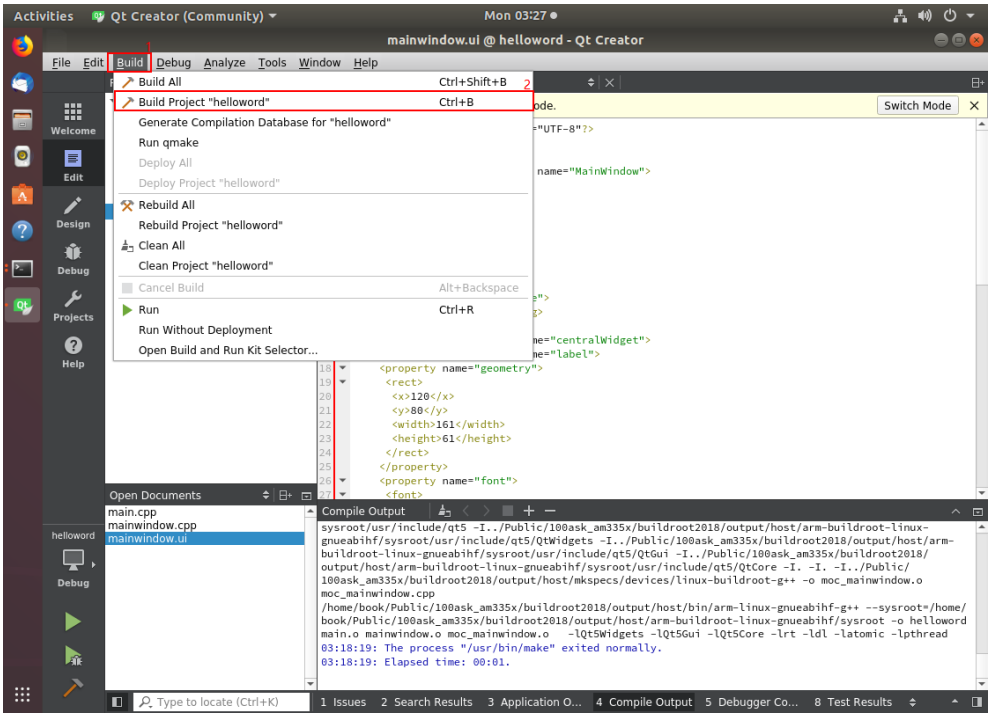


图 5-3-3 编译项目

3) 查看构建结果：

helloworld 项目构建成功后，编译好的二进制文件存放在以下目录中：

```
/home/book/build-helloword-100ask_am335x-Debug
```

可以使用 **file** 命令查看该 APP 是否被编译为 ARM 架构：

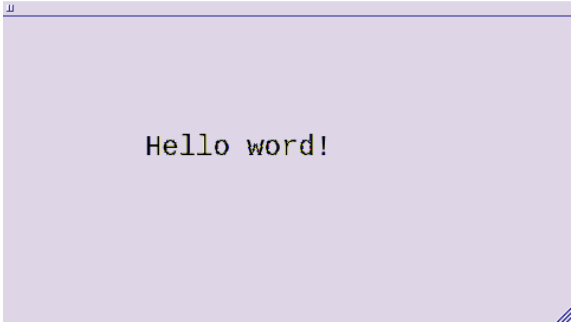
```
book@100ask:~/build-helloword-100ask_am335x-Debug$ file helloword
helloword: ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), dynamically linked,
interpreter /lib/ld-, for GNU/Linux 2.6.32,
BuildID[sha1]=bb5f7737070472574fc35af534b3e68e68bb02b6, with debug_info, not stripped
```

6.5 在开发板运行 Qt 程序

使用 ssh 远程登陆开发板，将生成的 QT 程序的可执行文件 helloworld 拷贝到开发板/usr/bin 目录下，然后在开发板上运行如下命令：

```
[root@100ask_am335x:~]#  helloworld
```

如果一切正常，将会在 LCD 屏幕上看到“Hello world!”的 Qt 窗口，如下图所示：



附录

常用网址

- 100ask_am335x 开发板 Wiki 地址 http://wiki.100ask.org/100ask_am335x
- 百问网淘宝店铺地址 https://100ask.taobao.com/
- 百问网论坛地址 http://bbs.100ask.org/forum.php
- 百问网技术支持邮箱账号 support@100ask.net
- 百问网 Github 站点地址 https://github.com/100askTeam
- 国内百问网 coding 站点地址 https://dev.tencent.com/u/weidongshan